PhyML – Manual

Version 3.0
November 5, 2026

https://github.com/stephaneguindon/phyml

http://www.atgc-montpellier.fr/phyml

Contents

1 Availability **5**

2 Authors **5**

3 Overview **6**

4 Bug report **6**

5 Installing PhyML **6**
5.1 Sources and compilation 6
5.2 Multithreading (OpenMP) 7
5.3 Running the test suite 7
5.4 Installing PhyML on UNIX-like systems (including Mac OS) . 8
5.5 Installing PhyML on Microsoft Windows 8
5.6 Installing the parallel version of PhyML 8
5.7 Installing PhyML-BEAGLE 9

6 Program usage. **9**
6.1 PHYLIP-like interface 9
6.1.1 Input Data sub-menu 10
6.1.2 Substitution model sub-menu 11
6.1.3 Tree searching sub-menu 13
6.1.4 Branch support sub-menu 15
6.2 Command-line interface 15
6.3 XML interface 23
6.4 Parallel bootstrap 23
6.5 Multithreading 24

7 Inputs & outputs for command-line and PHYLIP interface **25**
7.1 Sequence formats 25
7.1.1 Gaps and ambiguous characters 28
7.1.2 Specifying outgroup sequences 28
7.2 Tree format 29
7.3 Multiple alignments and trees 30
7.4 Custom amino-acid rate model 30
7.5 Topological constraint file 31
7.6 Output files 31
7.7 Treatment of invariable sites with fixed branch lengths 33

8 Model selection with SMS **33**

9 Protein models with several matrices **34**
9.1 LG4M and LG4X . . . 34
9.2 Models accounting for protein structure . . . 35

10 Inputs & outputs for the XML interface **36**
10.1 Mixture models in PhyML . . . 36
10.2 Partitionned (i.e., multiple-gene) analyses . . . 37
10.3 Combining mixture and partitions in PhyML: the theory . . . 40
10.4 The XML format and its use in PhyML . . . 41
10.5 Setting up mixture and partition models in PhyML: the basics 44
10.6 XML options . . . 46
10.6.1 `phyml` component . . . 46
10.6.2 `topology` component . . . 47
10.6.3 `ratematrices` component . . . 48
10.6.4 `equfreqs` component . . . 49
10.6.5 `branchlengths` component . . . 50
10.6.6 `siterates` component . . . 50
10.6.7 `partitionelem` and `mixtureelem` components . . . 52
10.7 Example: GTR + Γ4 + I . . . 53
10.8 Example: LG4X . . . 54
10.9 Example: CAT-like model . . . 57
10.10 Example: multiple partition elements . . . 58
10.11 Branch lengths with invariants and partionned data . . . 61

11 Citing PhyML **62**

12 Other programs **62**
12.1 PhyTime . . . 62
12.1.1 Installing PhyTime . . . 63
12.1.2 Running PhyTime . . . 63
12.1.3 PhyTime input . . . 63
12.1.4 Accounting for calibration uncertainty . . . 66
12.1.5 MCMC settings . . . 67
12.1.6 PhyTime output . . . 68
12.1.7 An example of PhyTime input and output files . . . 69
12.1.8 Citing PhyTime . . . 71
12.2 PhyloGeo . . . 73
12.2.1 Installing PhyloGeo . . . 73
12.2.2 Running PhyloGeo . . . 74
12.2.3 Citing PhyloGeo . . . 74
12.3 PhyREX . . . 74

12.3.1 Installing PhyREX 75
12.3.2 Running PhyREX 75
12.3.3 Citing PhyREX 80

13 Recommendations on program usage **81**
13.1 PhyML 81
13.2 PhyTime 82

14 Frequently asked questions **83**

15 Acknowledgements **84**

©Copyright 1999 - 2008 by PhyML Development Team.
The software PhyML is provided "as is" without warranty of any kind. In no event shall the authors or his employer be held responsible for any damage resulting from the use of this software, including but not limited to the frustration that you may experience in using the package. All parts of the source and documentation except where indicated are distributed under the GNU public licence. See http://www.opensource.org for details.

1 Availability

- Binaries: http://www.atgc-montpellier.fr/phyml
- Sources: http://stephaneguindon.github.io/phyml-downloads/
- Discussion forum: http://groups.google.com/group/phyml-forum

2 Authors

- Stéphane Guindon and Olivier Gascuel conceived the original PhyML algorithm.
- Stéphane Guindon conceived the PhyTime method.
- Stéphane Guindon, David Welch and Louis Ranjard conceived the PhyloGeo method.
- Stéphane Guindon, Wim Hordjik and Olivier Gascuel conceived the SPR-based tree search algorithm.
- Maria Anisimova and Olivier Gascuel conceived the aLRT method for branch support.
- Stéphane Guindon, Franck Lethiec, Jean-Francois Dufayard and Vincent Lefort implemented PhyML.
- Stéphane Guindon implemented PhyloGeo, PhyREX and PhyTime.
- Jean-Francois Dufayard created the benchmark and implemented the tools that are used to check PhyML accuracy and performances.
- Vincent Lefort, Stéphane Guindon, Patrice Duroux and Olivier Gascuel conceived and implemented PhyML web server.
- Imran Fanaswala interfaced PhyML with BEAGLE.
- Stéphane Guindon wrote this document.

3 Overview

PhyML [1] is a software package which primary task that is to estimate maximum likelihood phylogenies from alignments of nucleotide or amino-acid sequences. It provides a wide range of options that were designed to facilitate standard phylogenetic analyses. The main strength of PhyML lies in the large number of substitution models coupled to various options to search the space of phylogenetic tree topologies, going from very fast and efficient methods to slower but generally more accurate approaches. It also implements two methods to evaluate branch supports in a sound statistical framework (the non-parametric bootstrap and the approximate likelihood ratio test).

PhyML was designed to process moderate to large data sets. In theory, alignments with up to 4,000 sequences 2,000,000 character-long can analyzed. In practice however, the amount of memory required to process a data set is proportional of the product of the number of sequences by their length. Hence, a large number of sequences can only be processed provided that they are short. Also, PhyML can handle long sequences provided that they are not numerous. With most standard personal computers, the "comfort zone" for PhyML generally lies around 100-500 sequences less than 10,000 character long. For larger data sets, we recommend using other softwares such as RAxML [2] or GARLI [3] or Treefinder (http://www.treefinder.de).

4 Bug report

While PhyML is, of course, bug-free (!) (please read the disclaimer carefuly...), if you ever come across an issue, please feel free to report it using the discuss group web site at the following address: `https://groups.google.com/forum/?fromgroups#!forum/phyml-forum`. Alternatively, you can send an email to `s.guindon@auckland.ac.nz`. Do not forget to mention the version of PhyML and program options you are using.

5 Installing PhyML

5.1 Sources and compilation

The sources of the program are available free of charge from `http://stephaneguindon.github.io/phyml-downloads/`. The compilation on UNIX-like systems is fairly standard. It is described in the '`INSTALL`' file

that comes with the sources. In a command-line window, go to the directory that contains the sources and type:

```
./configure;
make clean;
make V=0;
```

By default, PhyML will be compiled with optimization flags turned on. It is possible to generate a version of PhyML that can run through a debugging tool (such as `ddd`) or a profiling tool (such as `gprof`) using the following instructions:

```
./configure --enable-debug;
make clean;
make V=0;
```

5.2 Multithreading (OpenMP)

PhyML can spread the likelihood computations over several threads of a single machine (option `--threads`, see Section 6.5). This feature relies on OpenMP, which `configure` enables automatically when the compiler supports it (`./configure --disable-openmp` turns it off). GCC supports OpenMP out of the box. With Apple's clang, install the `libomp` library (e.g., `brew install libomp`) and pass its location to `configure`:

```
./configure --enable-phyml \
  CPPFLAGS="-Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include" \
  LIBS="-L$(brew --prefix libomp)/lib -lomp"
make
```

5.3 Running the test suite

The `test/` directory contains a test suite that builds its own copies of PhyML from the sources (it does not need autotools) and checks them: C unit tests, and functional tests written in Python 3 (standard library only) that compare likelihoods with an independent implementation, check tree searches, branch supports, model selection and parameter estimation on simulated data, and run sanitizer, `-DNDEBUG`, OpenMP and MPI builds. Run it with:

```
make -C test check        # unit and functional tests (a few minutes)
make -C test check-mpi    # MPI bootstrap tests (needs mpicc and mpirun)
make -C test check-asan   # whole suite on the sanitizer build (slow)
```

Known bugs are tests reported as expected failures. The file `test/README.md` lists what is tested. GitHub Actions and GitLab CI run the suite on Linux (x86-64 and ARM64) and macOS.

5.4 Installing PhyML on UNIX-like systems (including Mac OS)

Copy PhyML binary file in the directory you like. For the operating system to be able to locate the program, this directory must be specified in the global variable `PATH`. In order to achieve this, you will have to add `export PATH="/your_path/:$PATH"` to the `.bashrc` or the `.bash_profile` located in your home directory (`your_path` is the path to the directory that contains PhyML binary).

5.5 Installing PhyML on Microsoft Windows

Copy the files `phyml.exe` and `phyml.bat` in the same directory. To launch PhyML, click on the icon corresponding to `phyml.bat`. Clicking on the icon for `phyml.exe` works too but the dimensions of the window will not fit PhyML PHYLIP-like interface.

5.6 Installing the parallel version of PhyML

Bootstrap analysis can run on multiple processors. Each processor analyses one bootstraped dataset. Therefore, the computing time needed to perform R bootstrap replicates is divided by the number of processors available.

This feature of PhyML relies on the MPI (Message Passing Interface) library. To use it, your computer must have MPI installed on it. In case MPI is not installed, you can dowload it from http://www.mcs.anl.gov/research/projects/mpich2/. Once MPI is installed, it is necessary to launch the MPI daemon. This can be done by entering the following instruction: `mpd &`. Note however that in most cases, the MPI daemon will already be running on your server so that you most likely do not need to worry about this. You can then just go in the `phyml/` directory (the directory that contains the `src/`, `examples/` and `doc/` folders) and enter the commands below:

```
./configure --enable-phyml-mpi;
make clean;
make;
```

A binary file named `phyml-mpi` has now been created in the `src/` directory and is ready to use with MPI. A typical MPI command-line which uses 4 CPUs is given below:

```
mpirun -n 4 ./phyml-mpi -i myseq -b 100
```

Please read section 6.4 of this document for more information.

5.7 Installing PhyML-BEAGLE

Note: I haven't found the time nor the resources to make sure the code of PhyML stays compatible with the BEAGLE library. Please do not hesitate at all to contact me if you'd like to contribute here.

PhyML can use the BEAGLE [4] library for the likelihood computation. BEAGLE provides provides significant speed-up: the single core version of PhyML-BEAGLE can be up to 10 times faster than PhyML on a single core and up to 150 times on Graphical Processing Units. PhyML-BEAGLE will eventually have of the features of PhyML, even though at the moment the boostrap and the invariant site options are not available. Also, please note that in some cases, the final log-likelihood reported by PhyML and PhyML-BEALGE may not exactly match, though the differences observed are very minor (in the 10^{-4} to 10^{-4} range).

In order to install PhyML-BEAGLE, you first need to download and install the BEAGLE library available from `https://code.google.com/p/beagle-lib/`. Then run the following commands:

```
./configure --enable-beagle;
make clean;
make;
```

A binary file named `phyml-beagle` will be created in the `src/` directory. The interface to `phyml-beagle` (i.e., commandline option of PHYLIP-like interface) is exactly identical to that of PhyML.

6 Program usage.

PhyML has three distinct user-interfaces. The first corresponds to a PHYLIP-like text interface that makes the choice of the options self-explanatory. The command-line interface is well-suited for people that are familiar with PhyML options or for running PhyML in batch mode. The XML interface is more sophisticated. It allows the user to analyse partitionned data using flexible mixture models of evolution.

6.1 PHYLIP-like interface

The default is to use the PHYLIP-like text interface by simply typing '`phyml`' in a command-line window or by clicking on the PhyML icon (see Section 5.5). After entering the name of the input sequence file, a list of sub-menus

helps the users set up the analysis. There are currently four distinct sub-menus:

1. *Input Data*: specify whether the input file contains amino-acid or nucleotide sequences. What the sequence format is (see Section 7) and how many data sets should be analysed.

2. *Substitution Model*: selection of the Markov model of substitution.

3. *Tree Searching*: selection of the tree topology searching algorithm.

4. *Branch Support*: selection of the method that is used to measure branch support.

'`+`' and '`-`' keys are used to move forward and backward in the sub-menu list. Once the model parameters have been defined, typing '`Y`' (or '`y`') launches the calculations. The meaning of some options may not be obvious to users that are not familiar with phylogenetics. In such situation, we strongly recommend to use the default options. As long as the format of the input sequence file is correctly specified (sub-menu *Input data*), the safest option for non-expert users is to use the default settings. The different options provided within each sub-menu are described in what follows.

6.1.1 Input Data sub-menu

```
[D] ............................  Data type (DNA/AA)
```

Type of data in the input file. It can be either DNA or amino-acid sequences in PHYLIP format (see Section 7). Type `D` to change settings.

```
[I] ......  Input sequences interleaved (or sequential)
```

PHYLIP format comes in two flavours: interleaved or sequential (see Section 7). Type `I` to selected among the two formats.

```
[M] ......................  Analyze multiple data sets
```

If the input sequence file contains more than one data sets, PhyML can analyse each of them in a single run of the program. Type `M` to change settings.

```
[R] ..........................................  Run ID
```

This option allows you to append a string that identifies the current PhyML run. Say for instance that you want to analyse the same data set with two models. You can then 'tag' the first PhyML run with the name of the first model while the second run is tagged with the name of the second model.

6.1.2 Substitution model sub-menu

```
[M] ................  Model of nucleotide substitution
```

```
[M] ................  Model of amino-acids substitution
```

PhyML implements a wide range of substitution models: JC69 [5], K80 [6], F81 [7], F84 [8], HKY85 [9], TN93 [10] GTR [11, 12] and custom for nucleotides; LG [13], WAG [14], Dayhoff [15], JTT [16], Blosum62 [17], mtREV [18], rtREV [19], cpREV [20], DCMut [21], VT [22] and mtMAM [23] and custom for amino acids. Cycle through the list of nucleotide or amino-acids substitution models by typing `M`. Both nucleotide and amino-acid lists include a 'custom' model. The custom option provides the most flexible way to specify the nucleotide substitution model. The model is defined by a string made of six digits. The default string is '`000000`', which means that the six relative rates of nucleotide changes: $A \leftrightarrow C$, $A \leftrightarrow G$, $A \leftrightarrow T$, $C \leftrightarrow G$, $C \leftrightarrow T$ and $G \leftrightarrow T$, are equal. The string '`010010`' indicates that the rates $A \leftrightarrow G$ and $C \leftrightarrow T$ are equal and distinct from $A \leftrightarrow C = A \leftrightarrow T = C \leftrightarrow G = G \leftrightarrow T$. This model corresponds to HKY85 (default) or K80 if the nucleotide frequencies are all set to 0.25. '`010020`' and '`012345`' correspond to TN93 and GTR models respectively. The digit string therefore defines groups of relative substitution rates. The initial rate within each group is set to 1.0, which corresponds to F81 (JC69 if the base frequencies are equal). Users also have the opportunity to define their own initial rate values. These rates are then optimised afterwards (option '`O`') or fixed to their initial values. The custom option can be used to implement all substitution models that are special cases of GTR. Table 1 on page 18 gives the correspondence between the 'standard' name of the model (see `http://mbe.oxfordjournals.org/content/18/6/897/F2.large.jpg`) and the custom model code. The custom model also exists for protein sequences. It is useful when one wants to use an amino-acid substitution model that is not hard-coded in PhyML. The symmetric part of the rate matrix, as well as the equilibrium amino-acid frequencies, are given in a file which name is given as input of the program. The format of this file is described in the section 7.4.

```
[F] ................  Optimise equilibrium frequencies
```

```
[E] .........  Equilibrium frequencies (empirical/user)
```

```
[F] . Amino acid frequencies (empirical/model defined)
```

For nucleotide sequences, optimising equilibrium frequencies means that the values of these parameters are estimated in the maximum likelihood framework. When the custom model option is selected, it is also possible to give the program a user-defined nucleotide frequency distribution at equilibrium (option `E`). For protein sequences, the stationary amino-acid frequencies are either those defined by the substitution model or those estimated by counting the number of different amino-acids observed in the data. Hence, the meaning of the `F` option depends on the type of the data to be processed.

```
[T] ....................  Ts/tv ratio (fixed/estimated)
```

Fix or estimate the transition/transversion ratio in the maximum likelihood framework. This option is only available when DNA sequences are to be analysed under K80, HKY85 or TN93 models. The definition given to this parameter by PhyML is the same as PAML's one. Therefore, the value of this parameter does *not* correspond to the ratio between the expected number of transitions and the expected number of transversions during a unit of time. This last definition is the one used in PHYLIP. PAML's manual gives more detail about the distinction between the two definitions (http://abacus.gene.ucl.ac.uk/software/paml.html).

```
[V] . Proportion of invariable sites (fixed/estimated)
```

The proportion of invariable sites, i.e., the expected frequency of sites that do not evolve, can be fixed or estimated. The default is to fix this proportion to 0.0. By doing so, we consider that each site in the sequence may accumulate substitutions at some point during its evolution, even if no differences across sequences are actually observed at that site. Users can also fix this parameter to any value in the $[0.0, 1.0]$ range or estimate it from the data in the maximum-likelihood framework.

```
[R] .......  One category of substitution rate (yes/no)
```

```
[C] ..........  Number of substitution rate categories
```

```
[A] ...  Gamma distribution parameter (fixed/estimated)
```

```
[G] .........'Middle' of each rate class (mean/median)
```

Rates of evolution often vary from site to site. This heterogeneity can be modelled using a discrete gamma distribution. Type `R` to switch this option on or off. The different categories of this discrete distribution correspond to different (relative) rates of evolution. The number of categories of this distribution is set to 4 by default. It is probably not wise to go below this number. Larger values are generally preferred. However, the computational burden involved is proportional to the number of categories (i.e., an analysis with 8 categories will generally take twice the time of the same analysis with only 4 categories). Note that the likelihood will not necessarily increase as the number of categories increases. Hence, the number of categories should be kept below a "reasonable" number, say 20. The default number of categories can be changed by typing `C`.

The middle of each discretized substitution rate class can be determined using the mean or the median. PAML, MrBayes and RAxML use the mean. However, the median is generally associated with greater likelihoods than the mean. This conclusion is based on our analysis of several real-world data sets extracted from TreeBase. Despite this, the default option in PhyML is to use the mean in order to make PhyML likelihoods comparable to those of other phylogenetic software. One must bare in mind that likelihoods calculated with the mean approximation are not directly comparable to the likelihoods calculated using the median approximation.

The shape of the gamma distribution determines the range of rate variation across sites. Small values, typically in the $[0.1, 1.0]$ range, correspond to large variability. Larger values correspond to moderate to low heterogeneity. The gamma shape parameter can be fixed by the user or estimated via maximum-likelihood. Type `A` to select one or the other option.

6.1.3 Tree searching sub-menu

```
[O] .........................  Optimise tree topology
```

By default the tree topology is optimised in order to maximise the likelihood. However, it is also possible to avoid any topological alteration. This option is useful when one wants to compute the likelihood of a tree given as input (see below). Type `O` to select among these two options.

```
[S] .................  Tree topology search operations
```

PhyML proposes three different methods to estimate tree topologies. The default approach is to use simultaneous NNI. This option corresponds to the original PhyML algorithm [1]. The second approach relies on subtree pruning and regrafting (SPR). It generally finds better tree topologies compared to NNI but is also significantly slower. The third approach, termed BEST, simply estimates the phylogeny using both methods and returns the best solution among the two. Type `S` to choose among these three choices.

```
[R] ........................  Use random starting tree
```

```
[N] .................  Number of random starting trees
```

When the SPR or the BEST options are selected, is is possible to use random trees rather than BioNJ or a user-defined tree, as starting tree. If this option is turned on (type `R` to change), five trees, corresponding to five random starts, will be estimated. The output tree file will contain the best tree found among those five. The number of random starts can be modified by typing `N`. Setting the number of random starting trees to N means that the analysis will take (slightly more than) N times the time required for a standard analysis where only one (BioNJ) starting tree is used. However, the analysis of real data sets shows that the best trees estimated using the random start option almost systematically have higher likelihoods than those inferred using a single starting tree.

```
[U] ........  Starting tree (BioNJ/parsimony/user tree)
```

When the tree topology optimisation option is turned on, PhyML proceeds by refining an input tree. By default, this input tree is estimated using BioNJ [24]. The alternative option is to use a parsimony tree. We found this option specially useful when analysing large data sets with NNI moves as it generally leads to greater likelihoods than those obtained when starting from a BioNJ trees. The user can also to input her/his own tree. This tree should be in Newick format (see Section 7). This option is useful when one wants to evaluate the likelihood of a given tree with a fixed topology, using PhyML. Type `U` to choose among these two options.

6.1.4 Branch support sub-menu

```
[B] ..............  Non parametric bootstrap analysis
```

The support of the data for each internal branch of the phylogeny can be estimated using non-parametric bootstrap. By default, this option is switched off. Typing `B` switches on the bootstrap analysis. The user is then prompted for a number of bootstrap replicates. The largest this number the more precise the bootstrap support estimates are. However, for each bootstrap replicate a phylogeny is estimated. Hence, the time needed to analyse N bootstrap replicates corresponds to N-times the time spent on the analysis of the original data set. $N = 100$ is generally considered as a reasonable number of replicates.

```
[A] ..............  Approximate likelihood ratio test
```

When the bootstrap option is switched off (see above), approximate likelihood branch supports are estimated. This approach is considerably faster than the bootstrap one. However, both methods intend to estimate different quantities and conducting a fair comparison between both criteria is not straightforward. The estimation of approximate likelihood branch support comes in multiple flavours. The default is set to aBayes, corresponding to the approximate Bayes method described in [25]. The approximate likelihood ratio test (aLRT) [26], Shimodaira–Hasegawa aLRT (SH-aLRT) statistics are the other available options.

6.2 Command-line interface

An alternative to the PHYLIP-like interface is the command-line interface. Users that do not need to modify the default parameters can launch the program with the '`phyml -i seq_file_name`' command. The list of all command line arguments and how to use them is given in the 'Help' section which is displayed when entering the '`phyml --help`' command. The available command-line options are described in what follows.

- `-i` (or `--input`) `seq_file_name`
  `seq_file_name` is the name of the nucleotide or amino-acid sequence file in PHYLIP, NEXUS or FASTA format (detected automatically).

- `-d` (or `--datatype`) `data_type`
  `data_type` is `nt` for nucleotide (default) and `aa` for amino-acid sequences.

- `-q` (or `--sequential`)
  Changes interleaved format (default) to sequential format.

- `-n` (or `--multiple`) `nb_data_sets`
  `nb_data_sets` is an integer giving the number of data sets to analyse.

- `-p` (or `--pars`)
  Use a minimum parsimony starting tree. This option is taken into account when the '-u' option is absent and when tree topology modifications are to be done.

- `-b` (or `--bootstrap`) `int`

  - `int` > 0: `int` is the number of bootstrap replicates.
  - `int` $= 0$: neither approximate likelihood ratio test nor bootstrap values are computed.
  - `int` = -1: approximate likelihood ratio test returning aLRT statistics.
  - `int` = -2: approximate likelihood ratio test returning Chi2-based parametric branch supports.
  - `int` = -4: SH-like branch supports alone.
  - `int` = -5: (default) approximate Bayes branch supports.

  With `int` > 0, the support of each branch is written in the output tree as the number of replicates (out of `int`) whose tree contains it, unless `--tbe` is set. The branch joining a sequence to an identical copy (duplicate sequences are removed before the analysis and put back in the final tree, see `--leave_duplicates`) has no support label: the supports are computed on the other sequences, where this branch does not exist.

- `--tbe`
  With `-b int` (> 0), computes the transfer bootstrap expectation (TBE, [27]) instead of the standard bootstrap proportions. A branch whose smaller side holds p taxa gets $1 - \bar{\phi}/(p-1)$, where $\bar{\phi}$ is the average over replicates of its transfer index, the number of taxa to move to find it in the replicate tree. TBE values lie between 0 and 1 and are less severe than bootstrap proportions for deep branches of large trees. The transfer index is computed in linear memory and takes milliseconds per replicate for thousands of taxa [28].

- `--bayesian_bootstrap`
  With `-b int` (> 0), each replicate reweights the sites with Dirichlet weights (Bayesian bootstrap) instead of resampling them. A branch counts as present in a replicate tree only if it is longer than 0.1 divided by the number of sites. Supports are written as counts, as for `-b`.

- `-m` (or `--model`) `model_name`
  `model_name` : substitution model name.

  - *Nucleotide-based models*: `HKY85` (default) | `JC69` | `K80` | `F81` | `F84` | `TN93` | `GTR` | `custom` | `r(ac),r(ag),r(at),r(cg),r(ct),r(gt)`
    JC90, K80, F81. F84, TN93 and GTR denote the "standard" substitution models. The `custom` option can be used to define a new substitution model. A string of six digits identifies the model. For instance, 000000 corresponds to F81 (or JC69 provided the distribution of nucleotide frequencies is uniform). 012345 corresponds to GTR. This option can be used for encoding any model that is a nested within GTR. See Section 6.1.2 and Table 1. *NOTE:* the substitution parameters of the custom model will be optimised so as to maximise the likelihood. Also, it is possible to specify and fix (i.e., avoid optimisation) the values of the substitution rates in the GTR model by passing the corresponding six rates (`r(ac),r(ag),r(at),r(cg),r(ct),r(gt)`) to the `-m` option, e.g., `-m 1.5,3,0.7,1.2,4,1`. Commas are used to separate the six relative rates and no blank space should be introduced here. These rates are then kept fixed.
  - *Amino-acid based models*: `LG` (default) | `WAG` | `JTT` | `MtREV` | `Dayhoff` | `DCMut` | `RtREV` | `CpREV` | `VT` | `AB` | `Blosum62` | `MtMam` | `MtArt` | `HIVw` | `HIVb` | `FLU` | `Q.pfam` | `Q.bird` | `Q.insect` | `Q.mammal` | `Q.plant` | `Q.yeast` | `MtZoa` | `custom`
    AB is the antibody model of [29], FLU the influenza model of [30], Q.pfam to Q.yeast the QMaker matrices of [31] and MtZoa the mitochondrial model of [32]. The `custom` option is useful when one wants to use an amino-acid substitution model that is not available by default in PhyML. The symmetric part of the rate matrix, as well as the equilibrium amino-acid frequencies, are given in a file which name is asked for by the program. The format of this file is described in section 7.4.
  - *Amino-acid models with several matrices*: `LG4M` | `LG4X` use one

Name	Command-line option
JC69	`-m 000000 -f 0.25,0.25,0.25,0.25`
F81	`-m 000000`
K80	`-m 010010 -f 0.25,0.25,0.25,0.25`
HKY85	`-m 010010`
TrNef	`-m 010020 -f 0.25,0.25,0.25,0.25`
TrN	`-m 010020`
K81	`-m 123321 -f 0.25,0.25,0.25,0.25`
K81uf	`-m 123321`
TIMef	`-m 132241 -f 0.25,0.25,0.25,0.25`
TIM	`-m 132241`
TVMef	`-m 102304 -f 0.25,0.25,0.25,0.25`
TVM	`-m 102304`
SYM	`-m 123456 -f 0.25,0.25,0.25,0.25`
GTR	`-m 123456`

Table 1. Nucleotide substitution model names (as defined in [35]) and the corresponding custom model code used in PhyML.

replacement matrix per rate category [33], and `EX | EX2 | EHO | EX_EHO` one matrix per class of solvent accessibility and/or secondary structure [34]. See Section 9.

- `--aa_rate_file file_name`
  This option is compulsory when analysing amino-acid sequences under a 'custom' model (see above). `file_name` should provide a rate matrix and equilibrium amino acid in PAML format (see Section 7.4).

- `-f e`, `m`, `o` or `fA,fC,fG,fT`
  Nucleotide or amino-acid frequencies.

  - `e` : the character frequencies are determined as follows :
    - *Nucleotide sequences*: (Empirical) the equilibrium base frequencies are estimated by counting the occurence of the different bases in the alignment.
    - *Amino-acid sequences*: (Empirical) the equilibrium amino-acid frequencies are estimated by counting the occurence of the different amino-acids in the alignment.
  - `m` : the character frequencies are determined as follows :

    - *Nucleotide sequences*: (Model) the equilibrium base frequencies are optimized using maximum likelihood.
    - *Amino-acid sequences*: (Model) the equilibrium amino-acid frequencies are estimated using the frequencies defined by the substitution model.
  - `o` : the character frequencies (nucleotides or amino-acids) are optimized using maximum likelihood.
  - `fA,fC,fG,fT`: only valid for nucleotide-based models. `fA`, `fC`, `fG` and `fT` are floating numbers that correspond to the frequencies of A, C, G and T respectively.

  Note: with amino-acid sequences and no `-f` option, the current version uses the empirical frequencies (`-f e`); use `-f m` to keep the frequencies of the substitution model. The models of Section 9 always use the frequencies of each of their matrices and do not accept `-f`.

- `-t` (or `--ts/tv`) `ts/tv_ratio`
  `ts/tv_ratio`: transition/transversion ratio. DNA sequences only. Can be a fixed positive value (e.g., 4.0) or type `e` to get the maximum likelihood estimate.

- `-v` (or `--pinv`) `prop_invar`
  `prop_invar`: proportion of invariable sites. Can be a fixed value in the [0,1] range or type `e` to get the maximum likelihood estimate.

- `-c` (or `--nclasses`) `nb_subst_cat`
  `nb_subst_cat`: number of relative substitution rate categories. Default: `nb_subst_cat=4`. Must be a positive integer.

- `-a` (or `--alpha`) `gamma`
  `gamma`: value of the gamma shape parameter. Can be a fixed positive value or e to get the maximum likelihood estimate. The value of this parameter is estimated in the maximum likelihood framework by default.

- `--use_median`
  The middle of each substitution rate class in the discrete gamma distribution is taken as the median. The mean is used by default.

- `--free_rates` or `--freerates`
  As an alternative to the discrete gamma model, it is possible to estimate the (relative) rate in each class of the (mixture) model and the

corresponding frequencies directly from the data. This model, called FreeRate, has more parameters than the discrete gamma one but usually provides a significantly better fit to the data. See [36] for more information about this model and an illustration of its use.

- `--il`
  `il` stands here for integrated (branch) length. This model, described in [37] in the context of molecular dating, provides an efficient way to implement the covarion model . Under the integrated length (IL) model, the length of each edge is described by a distribution of values, instead of a single value corresponding to the expected number of substitutions per position along the sequence. Let $l_{a,s}$ and $l_{b,s}$ be the number of substitutions at site s along edges a and b, and $l_{a,t}$ and $l_{b,t}$, the number of substitutions at site t. Standard models have $l_{a,s} = l_{a,t}$ and $l_{b,s} = l_{b,t}$, or $l_{a,s} = \alpha l_{a,t}$ and $l_{b,s} = \alpha l_{b,t}$ if rates vary across sites. The IL model has instead $l_{a,s} = \alpha l_{a,t}$ and $l_{b,s} = \beta l_{b,t}$ with $\alpha \neq \beta$, i.e. substitution rates vary across sites and edges. The IL approach is somehow an analytical approximation to the covarion model that, unlike the covarion model, does not incur any computational overhead compared to the traditional models. A notable difference with the plain vanilla covarion model and the IL model however is that substitution rates are not autocorrelated along the phylogeny under the IL model.

- `--codpos 1,2 or 3`
  When analysing an alignment of coding sequences, use this option to consider only the first, second or the third coding position for the estimation.

- `-s` (or `--search`) `move`
  Tree topology search operation option. Can be either `NNI` (default, fast) or `SPR` (usually slower than `NNI` but more accurate) or `BEST` (best of NNI and SPR search).

- `-u` (or `--inputtree`) `user_tree_file`
  `user_tree_file`: starting tree filename. The tree must be in Newick format.

- `-o params`
  This option focuses on specific parameter optimisation.

  - `params=tlr`: tree topology (`t`), branch length (`l`) and substitution rate parameters (`r`) are optimised.

  - `params=tl`: tree topology and branch lengths are optimised.
  - `params=lr`: branch lengths and substitution rate parameters are optimised.
  - `params=l`: branch lengths are optimised.
  - `params=r`: substitution rate parameters are optimised.
  - `params=n`: no parameter is optimised.

- `--rand_start`
  This option sets the initial tree to random. It is only valid if SPR searches are to be performed.

- `--n_rand_starts num`
  `num` is the number of initial random trees to be used. It is only valid if SPR searches are to be performed.

- `--r_seed num`
  `num` is the seed used to initiate the random number generator. Must be an integer.

- `--print_site_lnl`
  Print the likelihood for each site in file *_phyml_lk.txt. For Γ or Γ+I or FreeRate models, this option returns the posterior probability of each relative rate class at each site. Such information can then be used to identify fast- and slow-evolving regions of the alignment.

- `--print_trace`
  Print each phylogeny explored during the tree search process in file *_phyml_trace.txt. This option can be useful for monitoring the progress of the analysis for very large data sets and have an approximate idea of what the final phylogeny will look like.

- `--json_trace`
  Print each phylogeny explored during the tree search process in file *_phyml_json_trace.txt in JSON format (see http://www.json.org/). This option can be useful for monitoring the progress of the analysis for very large data sets and have an approximate idea of what the final phylogeny will look like.

- `--threads num`
  `num` is the number of threads used to compute likelihoods (default: 1). The work is split over site patterns, so it pays off for long alignments; results do not depend on the number of threads. Requires a build with OpenMP (Section 5.2). See Section 6.5.

- `--sms`
  Selects the substitution model with SMS [38] before the analysis, which then uses the selected model. See Section 8. Cannot be combined with `-m`, `-f`, `-v`, `-a`, `-t`, `--freerates` or `--aa_rate_file`.

- `--sms_criterion aic|bic`
  Criterion used by `--sms` to compare models (default: `bic`).

- `--sms_only`
  Runs the SMS model selection and stops, without inferring a tree.

- `--struct_model mix|part|conf_lg|conf_mix`
  With `-m EX`, `EHO` or `EX_EHO`: how the class matrices are combined (Section 9.2). Default: `conf_mix` with `--struct_file`, `mix` without.

- `--struct_file file_name`
  Site annotations (solvent accessibility and/or secondary structure) used by the `part`, `conf_lg` and `conf_mix` structure models. The format is described in Section 9.2.

- `--struct_chi value`, `--struct_props p1,p2,...`
  Fix the confidence coefficient χ or the class proportions of a structure model instead of estimating them.

- `--struct_fix_props`
  Keep the published class proportions in the `mix` and `conf_mix` structure models.

- `--run_id ID_string`
  Append the string ID_string at the end of each PhyML output file. This option may be useful when running simulations involving PhyML. It can also be used to 'tag' multiple analysis of the same data set with various program settings.

- `--no_memory_check`
  When the analysis needs more than 256 MB of memory and PhyML runs in a terminal, it asks the user to confirm that she/he wants to continue. The `--no_memory_check` option skips this question. When the standard input is not a terminal (batch jobs, pipes), PhyML only prints the estimate and goes on.

- `--no_colalias`
  By default, PhyML preprocesses each alignment by putting together (or aliasing) the columns that are identical. Use this option to skip

this step but be aware that the analysis might then take more time to complete.

- `--constrained_lens`
  When an input tree with branch lengths is provided, this option will find the branch multiplier that maximises the likelihood (i.e., the relative branch lengths remain constant)

- `--constraint_file file_name`
  `file_name` lists the topological constraints under which the tree topology search is conducted. This option should be used in conjunction with `-u file_name`. See Section 7.5 for more information.

- `--quiet`
  Runs PhyML in quiet mode. The program will not pause if the memory required to run the analysis exceeds 256MB and will not output the progression of the log-likelihood scores on the standard output.

- `--ancestral`
  PhyML calculates the marginal probabilities of each character state at each internal node and each site of the sequence alignment. It then uses the "Minimum Posterior Expected Error" criterion to infer ancestral sequences. Works for both nucleotide and amino-acid data.

- `--leave_duplicates`
  PhyML removes duplicate sequences by default. Use this option if you do not want PhyML to discard them (even though leaving these sequences in will slow down the analysis).

6.3 XML interface

- `--xml=xml_file_name`
  `xml_file_name` is the name of the XML file containing the information required to run the analysis. More details about this type of file is given in the section 10.

6.4 Parallel bootstrap

Bootstrapping is a highly parallelizable task. Indeed, bootstrap replicates are independent from one another. Each bootstrap replicate can then be analysed separately. Modern computers often have more than one CPU. Each CPU can therefore be used to process a bootstrap sample. Using this parallel

strategy, performing R bootstrap replicates on C CPUs 'costs' the same amount of computation time as processing $R \times C$ bootstrap replicates on a single CPU. In other words, for a given number of replicates, the computation time is divided by R compared to the non-parallel approach.

PhyML sources must be compiled with specific options to turn on the parallel option (see Section 5.6). Once the binary file (`phyml`) has been generated, running a bootstrap analysis with, say 100 replicates on 2 CPUs, can be done by typing the following command-line:

```
mpd &;
mpirun -np 2 ./phyml-mpi -i seqfile -b 100;
```

The first command launches the mpi daemon while the second launches the analysis. Note that launching the daemon needs to be done only once. The output files are similar to the ones generated using the standard, non-parallel, analysis (see Section 7). Note that running the program in batch mode, i.e.:

```
mpirun -np 2 ./phyml-mpi -i seqfile -b 100 &
```

will probably NOT work. I do not know how to run a mpi process in batch mode yet. Suggestions welcome... Also, at the moment, the number of bootstrap replicates must be a multiple of the number of CPUs required in the mpirun command.

6.5 Multithreading

With a build that supports OpenMP (Section 5.2), `--threads num` computes the likelihood with `num` threads, each handling a share of the site patterns (at least 128 patterns per thread). The output files are identical whatever the number of threads, so analyses remain reproducible. Multithreading speeds up long alignments; for bootstrap analyses, the parallel version based on MPI (Section 6.4) distributes the replicates instead, and the two can be combined. Mixture and partition models defined in XML still run on one thread. A build with OpenMP runs as fast on one thread as a build without it. On alignments of about a thousand site patterns, the gain flattens beyond four threads (about 2.8 to 3 times faster with eight threads on an AMD EPYC node), because each thread then has little work per likelihood evaluation; with more site patterns, each thread gets more work.

```
phyml -i seqfile -d aa -m LG --threads 8
```

PHYLIP interleaved

```
5 80
seq1  CCATCTCACGGTCGGTACGATACACCKGCTTTTGGCAGGAAATGGTCAATATTACAAGGT
seq2  CCATCTCACGGTCAG---GATACACCKGCTTTTGGCGGGAAATGGTCAACATTAAAAGAT
seq3  RCATCTCCCGCTCAG---GATACCCCKGCTGTTG??????????????ATTAAAAGGT
seq4  RCATCTCATGGTCAA---GATACTCCTGCTTTTGGCGGGAAATGGTCAATCTTAAAAGGT
seq5  RCATCTCACGGTCGGTAAGATACACCTGCTTTTGGCGGGAAATGGTCAAT????????GT

ATCKGCTTTTGGCAGGAAAT
ATCKGCTTTTGGCGGGAAAT
AGCKGCTGTTG??????????
ATCTGCTTTTGGCGGGAAAT
ATCTGCTTTTGGCGGGAAAT
```

PHYLIP sequential

```
5 40
seq1  CCATCTCANNNNNNNNNACGATACACCKGCTTTTGGCAGG
seq2  CCATCTCANNNNNNNNNGGGATACACCKGCTTTTGGCGGG
seq3  RCATCTCCCGCTCAGTGAGATACCCCKGCTGTTGXXXXX
seq4  RCATCTCATGGTCAATG-AATACTCCTGCTTTTGXXXXX
seq5  RCATCTCACGGTCGGTAAGATACACCTGCTTTTGxxxxx
```

Figure 1. **PHYLIP interleaved and sequential formats.**

7 Inputs & outputs for command-line and PHYLIP interface

PhyML reads data from standard text files, without the need for any particular file name extension.

7.1 Sequence formats

Alignments of DNA or protein sequences must be in PHYLIP or NEXUS [39] sequential or interleaved format (Figures 7.1 and 2). For PHYLIP formated sequence alignments, the first line of the input file contains the number of species and the number of characters, in free format, separated by blank characters. One slight difference with PHYLIP format deals with sequence name lengths. While PHYLIP format limits this length to ten characters, PhyML can read up to hundred character long sequence names. Blanks and the symbols "(),:" are not allowed within sequence names because the Newick tree format makes special use of these symbols. Another slight difference with PHYLIP format is that actual sequences must be separated from their names by at least one blank character.

A PHYLIP input sequence file may also display more than a single data set. Each of these data sets must be in PHYLIP format and two successive alignments must be separated by an empty line. Processing multiple data

Nexus nucleotides

```
[ This is a comment ]
#NEXUS
BEGIN DATA;
DIMENSIONS NTAX=10 NCHAR=20;
FORMAT DATATYPE=DNA;
MATRIX
tax1        ?ATGATTTCCTTAGTAGCGG
tax2        CAGGATTTCCTTAGTAGCGG
tax3        ?AGGATTTCCTTAGTAGCGG
tax4        ????????????GTAGCGG
tax5        CAGGATTTCCTTAGTAGCGG
tax6        CAGGATTTCCTTAGTAGCGG
tax7        ???GATTTCCTTAGTAGCGG
tax8        ????????????????????
tax9        ???GGATTTCTTCGTAGCGG
tax10       ????????????????AGCGG;
END;
```

Nexus digits

```
[ This is a comment ]
#NEXUS
BEGIN DATA;
DIMENSIONS NTAX=10 NCHAR=20;
FORMAT DATATYPE=STANDARD SYMBOLS="0 1 2 3";
MATRIX
tax1        ?03203331133023021 22
tax2        10220333113302302122
tax3        ?02203331133023021 22
tax4        ????????????2302122
tax5        10220333113302302122
tax6        10220333113302302122
tax7        ???20333113302302122
tax8        ????????????????????
tax9        ???22033313312302122
tax10       ????????????????02122;
END;
```

Nexus digits

```
[ This is a comment ]
#NEXUS
BEGIN DATA;
DIMENSIONS NTAX=10 NCHAR=20;
FORMAT DATATYPE=STANDARD SYMBOLS="00 01 02 03";
MATRIX
tax1        ??0003020003030301010303000203000201020 2
tax2        0100020200030303010103030002030002010202
tax3        ??0002020003030301010303000203000201020 2
tax4        ????????????????????????????02030002010202
tax5        0100020200030303010103030002030002010202
tax6        0100020200030303010103030002030002010202
tax7        ??????020003030301010303000203000201020 2
tax8        ????????????????????????????????????????
tax9        ??????020200030303010303010203000201020 2
tax10       ????????????????????????????????0002010202;
END;
```

Figure 2. **NEXUS formats.**

sets requires to toggle the 'M' option in the *Input Data* sub-menu or use the '-n' command line option and enter the number of data sets to analyse. The multiple data set option can be used to process re-sampled data that were generated using a non-parametric procedure such as cross-validation or jackknife (a bootstrap option is already included in PhyML). This option is also useful in multiple gene studies, even if fitting the same substitution model to all data sets may not be suitable.

PhyML also reads aligned sequences in FASTA format, detected when the first non-blank character of the file is '>'. Each sequence starts with a line >name; the name is the first word after '>', and the rest of the line (a description) is ignored. The sequence follows on one or more lines, up to the next line starting with '>'. All the sequences must have the same length, blank characters and line breaks (including Windows line endings) are ignored, and '.' is read as a gap, as in most FASTA alignments. Names follow the same rules as in PHYLIP files. A FASTA file holds a single data set, so the -n option cannot be used with it, and -q does not apply.

FASTA

```
>seq1 any description
CCATCTCACGGTCGGTACGATACACCKGCTTTTGGCAGGAAATGGTCAATATTACAAGGT
ATCKGCTTTTGGCAGGAAAT
>seq2
CCATCTCACGGTCAG---GATACACCKGCTTTTGGCGGGAAATGGTCAACATTAAAAGAT
ATCKGCTTTTGGCGGGAAAT
>seq3
RCATCTCCCGCTCAG---GATACCCCKGCTGTTG????????????????ATTAAAAGGT
AGCKGCTGTTG??????????
```

PhyML can also process alignments in NEXUS format. Although not all the options provided by this format are supported by PhyML, a few specific features are exploited. Of course, this format can handle nucleotide and protein sequence alignments in sequential or interleaved format. It is also possible to use custom alphabets, replacing the standard 4-state and 20-state alphabets for nucleotides and amino-acids respectively. Examples of a 4-state custom alphabet are given in Figure 2. Each state must here correspond to one digit or more. The set of states must be a list of consecutive digits starting from 0. For instance, the list "0, 1, 3, 4" is not a valid alphabet. Each state in the symbol list must be separated from the next one by a space. Hence, alphabets with large number of states can be easily defined by using two-digit number (starting with 00 up to 19 for a 20 state alphabet). Most importantly, this feature gives the opportunity to analyse data sets made of presence/absence character states (use the symbols=''0 1'' option for such data). Alignments made of custom-defined states will be processed using the Jukes and Cantor model. Other options of the program (e.g., number of rate

Character	Nucleotide	Character	Nucleotide
A	Adenosine	Y	C or T
G	Guanosine	K	G or T
C	Cytidine	B	C or G or T
T	Thymidine	D	A or G or T
U	Uridine ($=T$)	H	A or C or T
M	A or C	V	A or C or G
R	A or G	$-$ or N or X or ?	unknown
W	A or T		($=A$ or C or G or T)
S	C or G		

Table 2. **List of valid characters in DNA sequences and the corresponding nucleotides.**

classes, tree topology search algorithm) are freely configurable. Note that, at the moment, the maximum number of different states is set to 22 in order to save memory space. It is however possible to lift this threshold by modifiying the value of the variable `T_MAX_ALPHABET` in the file '`utilities.h`'. The program will then have to be re-compiled.

7.1.1 Gaps and ambiguous characters

Gaps correspond to the '`-`' symbol. They are systematically treated as unknown characters "on the grounds that we don't know what would be there if something were there" (J. Felsenstein, PHYLIP main documentation). The likelihood at these sites is summed over all the possible states (i.e., nucleotides or amino acids) that could actually be observed at these particular positions. Note however that columns of the alignment that display only gaps or unknown characters are simply discarded because they do not carry any phylogenetic information (they are equally well explained by any model). PhyML also handles ambiguous characters such as R for A or G (purines) and Y for C or T (pyrimidines). Tables 2 and 3 give the list of valid characters/symbols and the corresponding nucleotides or amino acids.

7.1.2 Specifying outgroup sequences

PhyML can return rooted trees provided outgroup taxa are identified from the sequence file. In order to do so, sequence names that display a '*' character will be automatically considered as belonging to the outgroup.

The topology of the rooted tree is exactly the same as the unrooted version

Character	Amino-Acid	Character	Amino-Acid
A	Alanine	*L*	Leucine
R	Arginine	*K*	Lysine
N or *B*	Asparagine	*M*	Methionine
D	Aspartic acid	*F*	Phenylalanine
C	Cysteine	*P*	Proline
Q or *Z*	Glutamine	*S*	Serine
E	Glutamic acid	*T*	Threonine
G	Glycine	*W*	Tryptophan
H	Histidine	*Y*	Tyrosine
I	Isoleucine	*V*	Valine
L	Leucine	− or *X* or ?	unknown
K	Lysine		(can be any amino acid)

Table 3. **List of valid characters in protein sequences and the corresponding amino acids.**

of the same tree. In other words, PhyML first ignores the distinction between ingroup and outgroup sequences, builds a maximum likelihood unrooted tree and then tries to add the root. If the outgroup has more than one sequence, the position of the root might be ambiguous. In such situation, PhyML tries to identify the most relevant position of the root by considering which edge provides the best separation between ingroup and outgroup taxa (i.e., we are trying to make the outgroup "as monophyletic as possible").

7.2 Tree format

PhyML can read one or several phylogenetic trees from an input file. This option is accessible through the *Tree Searching* sub menu or the '`-u`' argument from the command line. Input trees are generally used as initial maximum likelihood estimates to be subsequently adjusted by the tree searching algorithm. Trees can be either rooted or unrooted and multifurcations are allowed. Taxa names must, of course, match the corresponding sequence names.

```
((seq1:0.03,seq2:0.01):0.04,(seq3:0.01,(seq4:0.2,seq5:0.05):0.2):0.01);
((seq3,seq2),seq1,(seq4,seq5));
```

Figure 3. **Input trees**. The first tree (top) is rooted and has branch lengths. The second tree (bottom) is unrooted and does not have branch lengths.

7.3 Multiple alignments and trees

Single or multiple sequence data sets may be used in combination with single or multiple input trees. When the number of data sets is one ($n_D = 1$) and there is only one input tree ($n_T = 1$), then this tree is simply used as input for the single data set analysis. When $n_D = 1$ and $n_T > 1$, each input tree is used successively for the analysis of the single alignment. PhyML then outputs the tree with the highest likelihood. If $n_D > 1$ and $n_T = 1$, the same input tree is used for the analysis of each data set. The last combination is $n_D > 1$ and $n_T > 1$. In this situation, the i-th tree in the input tree file is used to analyse the i-th data set. Hence, n_D and n_T must be equal here.

7.4 Custom amino-acid rate model

The custom amino-acid model of substitutions can be used to implement a model that is not hard-coded in PhyML. This model must be time-reversible. Hence, the matrix of substitution rates is symmetrical. The format of the rate matrix with the associated stationary frequencies is identical to the one used in PAML. An example is given below:

```
0.55
0.51  0.64
0.74  0.15  5.43
1.03  0.53  0.27  0.03
0.91  3.04  1.54  0.62  0.10
1.58  0.44  0.95  6.17  0.02  5.47
1.42  0.58  1.13  0.87  0.31  0.33  0.57
0.32  2.14  3.96  0.93  0.25  4.29  0.57  0.25
0.19  0.19  0.55  0.04  0.17  0.11  0.13  0.03  0.14
0.40  0.50  0.13  0.08  0.38  0.87  0.15  0.06  0.50  3.17
0.91  5.35  3.01  0.48  0.07  3.89  2.58  0.37  0.89  0.32  0.26
0.89  0.68  0.20  0.10  0.39  1.55  0.32  0.17  0.40  4.26  4.85  0.93
0.21  0.10  0.10  0.05  0.40  0.10  0.08  0.05  0.68  1.06  2.12  0.09  1.19
1.44  0.68  0.20  0.42  0.11  0.93  0.68  0.24  0.70  0.10  0.42  0.56  0.17  0.16
3.37  1.22  3.97  1.07  1.41  1.03  0.70  1.34  0.74  0.32  0.34  0.97  0.49  0.55  1.61
2.12  0.55  2.03  0.37  0.51  0.86  0.82  0.23  0.47  1.46  0.33  1.39  1.52  0.17  0.80  4.38
0.11  1.16  0.07  0.13  0.72  0.22  0.16  0.34  0.26  0.21  0.67  0.14  0.52  1.53  0.14  0.52  0.11
0.24  0.38  1.09  0.33  0.54  0.23  0.20  0.10  3.87  0.42  0.40  0.13  0.43  6.45  0.22  0.79  0.29  2.49
2.01  0.25  0.20  0.15  1.00  0.30  0.59  0.19  0.12  7.82  1.80  0.31  2.06  0.65  0.31  0.23  1.39  0.37  0.31

8.66  4.40  3.91  5.70  1.93  3.67  5.81  8.33  2.44  4.85  8.62  6.20  1.95  3.84  4.58  6.95  6.10  1.44  3.53  7.09
```

The entry on the i-th row and j-th column of this matrix corresponds to the rate of substitutions between amino-acids i and j. The last line in the file gives the stationary frequencies and must be separated from the rate matrix by one line. The ordering of the amino-acids is alphabetical, i.e, Ala, Arg, Asn, Asp, Cys, Gln, Glu, Gly, His, Ile, Leu, Lys, Met, Phe, Pro, Ser, Thr, Trp, Tyr and Val.

7.5 Topological constraint file

PhyML can perform phylogenetic tree estimation under user-specified topological constraints. In order to do so, one should use the `--constraint_file file_name` command-line option where `file_name` lists the topological constraints. Such constraints are straightforward to define. For instance, the following constraints:

```
((A,B),C,(D,E,F));
```

indicate that taxa D, E and F belong to the same clade. A, B and C also belong to the same clade and the two clades hence defined should not overlap. Under these two constraints, the tree ((A,B),D,((E,F),C)) is not valid. From the example above, you will notice that the constraints are defined using a multifurcating tree in NEWICK format. Note that this tree does not need to display the whole list of taxa. For instance, while the only taxa involved in specifying topological constraints above are A, B, C, D, E & F, the actual data set could include more than these six taxa only.

A constraint is satisfied when the estimated tree, restricted to the taxa of the constraint tree, displays every clade of the constraint tree: with the constraint `((t1,t3),t2,t4)`, the estimated tree must separate t1 and t3 from t2 and t4, whatever the position of the other taxa. The search starts from the constraint tree itself: its polytomies are resolved arbitrarily, the taxa it does not name are attached to its root, and its branch lengths are estimated with BioNJ (a tree given with `-u` is not used). Every tree topology move (NNI or SPR) that would break a constraint is rejected, so the output tree complies with the constraints.

7.6 Output files

Table 4 presents the list of files resulting from an analysis. Basically, each output file name can be divided into three parts. The first part is the sequence file name, the second part corresponds to the extension '`_phyml_`' and the third part is related to the file content. When launched with the default options, PhyML only generates two files: the tree file and the model parameter file. The estimated maximum likelihood tree is in standard Newick format (see Figure 3). The model parameters file, or statistics file, displays the maximum likelihood estimates of the substitution model parameters, the likelihood of the maximum likelihood phylogenetic model, and other important information concerning the settings of the analysis (e.g., type of data, name of the substitution model, starting tree, etc.). Two additional output files are created if bootstrap supports were evaluated. These files simply

Sequence file name : 'seq'

Output file name	Content
seq_phyml_tree.txt	ML tree
seq_phyml_stats.txt	ML model parameters
seq_phyml_boot_trees.txt	ML trees – bootstrap replicates
seq_phyml_boot_stats.txt	ML model parameters – bootstrap replicates
seq_phyml_rand_trees.txt	ML trees – multiple random starts
seq_phyml_ancestral_seq.txt	ancestral sequences
seq_phyml_ancestral_tree.txt	ML tree with node labels as in ancestral sequence file
seq_phyml_sms.csv	fits of the models compared by SMS (option --sms)

Table 4. **Standard output files**

contain the maximum likelihood trees and the substitution model parameters estimated from each bootstrap replicate. Such information can be used to estimate sampling errors around each parameter of the phylogenetic model. When the random tree option is turned on, the maximum likelihood trees estimated from each random starting trees are printed in a separate tree file (see last row of Table 4).

PhyML estimates ancestral sequences by calculating the *marginal* (as opposed to the *joint*) probability of each character state at each internal node of the phylogeny. These probabilities are given in the file seq_phyml_ancestral_seq.txt. The bulk of this file is a table where each row corresponds to a site in the original alignment and a number corresponding labeling each internal node. The different columns of that file give the probability of each character state given the data observed at the corresponding sites and the estimated phylogeny. PhyML also outputs an extra column called MPEE, which stands for Minimum Posterior Expected Error. This column gives the state, which can be a single nucleotide/amino-acid or a combination of more than one nucleotide/amino-acid, that is optimal under the MPEE criterion [40].

Also, it is relatively straightforward to identify which number corresponds to which node in the tree by examining the information provided in the Newick-formatted file seq_phyml_ancestral_tree.txt.

7.7 Treatment of invariable sites with fixed branch lengths

PhyML allows users to give an input tree with fixed topology and branch lengths and find the proportion of invariable sites that maximise the likelihood (option `-o r`). These two options can be considered as conflicting since branch lengths depend on the proportion of invariants. Hence, changing the proportion of invariants implies that branch lengths are changing too. More formally, let l denote the length of a branch, i.e., the expected number of substitutions per site, and p be the proportion of invariants. We have $l = (1-p)l'$, where l' is the expected number of substitutions *at variable sites*. When asked to optimize p but leave l unchanged, PhyML does the following:

1. Calculate $l' = l/(1-p)$ and leave l' unchanged throughout the optimization.

2. Find the value of p that maximises the likelihood. Let p^* denote this value.

3. Set $l^* = (1-p^*)l'$ and print out the tree with l^* (instead of l).

PhyML therefore assumes that the users wants to fix the branch lengths measured at *variable* sites only (i.e., l^* is fixed). This is the reason why the branch lengths in the input and output trees do differ despite the use of the the `-o r` option. While we believe that this approach relies on a sound rationale, it is not perfect. In particular, the original transformation of branch lengths ($l' = l/(1-p)$) relies on a default value for p with is set to 0.2 in practice. It is difficult to justify the use of this value rather than another one. One suggestion proposed by Bart Hazes is to avoid fixing the branch lengths altogether and rather estimate the value of a scaling factor applied to each branch length in the input tree (option `--contrained_lens`). We agree that this solution probably matches very well most users expectation, i.e., "find the best value of p while constraining the ratio of branch lengths to be that given in the input tree". Please feel free to send us your suggestions regarding this problem by posting on the forum (http://groups.google.com/group/phyml-forum).

8 Model selection with SMS

SMS (Smart Model Selection, [38]) selects the substitution model before the analysis. It used to be a set of scripts running PhyML once per candidate

model; it is now built in (`--sms`). The candidate models are fitted on a fixed tree, in four steps:

1. a BioNJ tree (or the tree given with `-u`) is built and its branch lengths estimated with GTR+Γ+I (nucleotides) or LG (amino acids);
2. the best rate decoration is chosen among none, +Γ, +I, +Γ+I and +R (FreeRate);
3. the best matrix is chosen: GTR, TN93, HKY85 and K80 for nucleotides; for amino acids, the 23 built-in matrices, with and without +F, are tried in the order of the chi-square distance between their frequencies and those of the alignment;
4. the decoration is chosen again for the selected matrix.

Models are compared with the BIC (default) or the AIC (`--sms_criterion aic`). The analysis then uses the selected model, starting from its SMS estimates. The fits are written to `seq_phyml_sms.csv` (one row per model: number of parameters, log-likelihood, AIC and BIC, and the order in which the models were tried). `-c` sets the number of rate categories of +Γ and +R (default 4); `--sms_only` stops after the selection.

```
phyml -i seqfile -d aa --sms
phyml -i seqfile --sms --sms_criterion aic --sms_only
```

9 Protein models with several matrices

9.1 LG4M and LG4X

LG4M and LG4X [33] use a different replacement matrix for each rate category, from the slowest (M1 or X1) to the fastest (M4 or X4). Each matrix Q_k has its own exchangeabilities and amino-acid frequencies π_k and is normalised to one substitution per unit of time. With x_s the data at site s and $L(x_s \mid Q, \pi, \rho)$ the likelihood of the tree with all branch lengths multiplied by ρ:

- `-m LG4M`: four Gamma categories of equal weights, the k-th of which uses matrix M_k,

$$L_s = \frac{1}{4}\sum_{k=1}^{4} L\big(x_s \mid Q_k^M, \pi_k^M, \gamma_k(\alpha)\big),$$

  where $\gamma_k(\alpha)$ is the mean rate of the k-th category. The only free parameter is α (estimated unless `-a` gives its value).

- `-m LG4X`: four categories with free weights w_k and rates ρ_k,

$$L_s = \sum_{k=1}^{4} w_k \, L\big(x_s \mid Q_k^X, \pi_k^X, \rho_k\big), \qquad \sum_k w_k = 1, \quad \sum_k w_k \rho_k = 1,$$

  that is, six free parameters. As in [33], the tree and α are first estimated with LG4M; LG4X then starts from equal weights and the Gamma rates, and estimates its weights and rates. Matrix X_k stays with category k even if its rate is not the k-th one.

The matrices are those published by the authors (http://www.atgc-montpellier.fr/models/lg4x). Following [33], these models have four rate categories and neither invariant sites nor estimated frequencies: `-f`, `-v`, `--freerates`, `-c` other than 4 and `--use_median` are refused. The statistics file gives the rate and weight of each matrix. LG4M runs about as fast as LG.

```
phyml -i seqfile -d aa -m LG4X
```

9.2 Models accounting for protein structure

The models of [34] use one replacement matrix per class of sites: buried or exposed to the solvent (`-m EX`, also written `EX2`), extended, helix or other secondary structure (`-m EHO`), or the six combinations (`-m EX_EHO`). Each class k has its own matrix Q_k, frequencies π_k and global rate r_k, and rates vary across sites according to a discrete Gamma distribution with G categories (`-c`, `-a`). With

$$L_k(s) = \frac{1}{G} \sum_{g=1}^{G} L\big(x_s \mid Q_k, \pi_k, r_k \, \gamma_g\big),$$

`--struct_model` chooses how the classes are combined at site s, whose annotated class is a_s:

- `mix`: a mixture, without annotations, $L_s = \sum_k P_k \, L_k(s)$, where the class proportions P_k are estimated;
- `part`: each site uses the matrix of its class, $L_s = L_{a_s}(s)$;
- `conf_lg`: the annotation is trusted with probability χ, otherwise the site follows LG, $L_s = \chi \, L_{a_s}(s) + (1 - \chi) \, L_{LG}(s)$;

- `conf_mix`: the annotation is trusted with probability χ, otherwise the site follows the mixture, $L_s = \chi\, L_{a_s}(s) + (1-\chi) \sum_k P_k\, L_k(s)$.

χ is estimated (`--struct_chi` fixes it), as are the proportions P_k of `mix` and `conf_mix` (`--struct_props` gives their values, `--struct_fix_props` keeps the published ones). A site without annotation follows the mixture (LG with `conf_lg`). The matrices, rates and class proportions are those of the authors (http://www.atgc-montpellier.fr/phyml/structure/). The output tree gives branch lengths in substitutions per site: they are multiplied by the mean rate of the model. `-f`, `-v`, `--freerates` and `--aa_rate_file` cannot be used with these models.

The annotations are given by `--struct_file`, a text file with one line per kind of annotation and one character per alignment site:

```
# solvent accessibility (B buried, E exposed)
exposure:  BBEEBEE?EB...
# secondary structure (E extended, H helix, O other)
structure: HHHOOEE-EH...
```

Unknown values are written `?`, `X` or `-`. The other DSSP states (G, I, B, S, T, C) count as 'other'. `EX` needs the `exposure` line, `EHO` the `structure` line, and `EX_EHO` both. Lines starting with `#` are ignored.

```
phyml -i seqfile -d aa -m EX_EHO --struct_model conf_mix \
      --struct_file annot.txt
```

10 Inputs & outputs for the XML interface

10.1 Mixture models in PhyML

PhyML implements a wide range of mixture models. The discrete gamma model [41] is arguably the most popular of these models in phylogenetics. However, in theory, mixture models are not restricted to the description of the variation of substitution rates across sites. For instance, if there are good reasons to believe that the relative rates of substitution between nucleotides vary along the sequence alignments, it makes sense to use a mixture of GTR models. Consider the case where substitutions between A and C occur at high rate in some regions of the alignment and low rate elsewhere, a mixture with two classes, each class having its own GTR rate matrix, would be suitable. The likelihood at any site of the alignment is then obtained by averaging the likelihoods obtained for each GTR rate matrix, with the same weight given to each of these matrices.

PhyML implements a generic framework that allows users to define mixtures on substitution rates, rate matrices and nucleotide or amino-acid equilibrium frequencies. Each class of the mixture model is built by assembling a substitution rate, a rate matrix[1] and a vector of equilibrium frequencies. For instance, let $\{R_1, R_2, R_3\}$ be a set of substitution rates, $\{M_1, M_2\}$ a set of rate matrices and $\{F_1, F_2\}$ a set of vectors of equilibrium frequencies. One could then define the first class of the mixture model as $\mathcal{C}_1 = \{R_1, M_1, F_1\}$, a second class as $\mathcal{C}_2 = \{R_2, M_1, F_1\}$, and a third class as $\mathcal{C}_3 = \{R_3, M_2, F_2\}$. If R_1, R_2 and R_3 correspond to slow, medium and fast substitution rates, then this mixture model allows the fast evolving rates to have their own vector of equilibrium frequencies and rate matrix, distinct from that found at the medium or slow evolving sites. The likelihood at any given site D_s of the alignment is then:

$$\Pr(D_s) = \sum_{c=1}^{3} \Pr(D_s|\mathcal{C}_s = c) \Pr(\mathcal{C}_s = c),$$

where $\Pr(\mathcal{C}_s = c)$ is obtained by multiplying the probability (density) of the three components (i.e., rate, matrix, frequencies). For instance, $\Pr(\mathcal{C}_1 = \{R_1, M_1, F_1\}) = \Pr(R_1) \times \Pr(M_1) \times \Pr(F_1)$. We therefore assume here that substitution rates, rate matrices and equilibrium frequencies are independent from one another.

Note that, using the same substitution rates, rate matrices and vector of equilibrium frequencies, it is possible to construct many other mixture models. For instance, the mixture model with the largest number of classes can be created by considering all the combinations of these three components. We would then get a mixture of $3 \times 2 \times 2 = 12$ classes, corresponding to all the possible combinations of 3 rates, 2 matrices and 2 vectors of frequencies.

10.2 Partitionned (i.e., multiple-gene) analyses

We first introduce some terms of vocabulary that have not been presented before. A partitionned data set, also referred to as partition, is a set of partition elements. Typically, a partitionned data set will be made of a set of distinct gene alignments. A partition element will then correspond to one (or several) of these gene alignments. Note that the biology litterature often uses the term partition to refer to an element of a partitionned data. We thus use here instead the mathematical definition of the terms 'partition' and 'partition element'.

[1] the rate matrix corresponds here the symmetrical matrix giving the so-called "echangeability rates"

Phylogenetics models usually assume individual columns of an alignment to evolve independently from one another. Codon-based models (e.g., [42–45]) are exceptions to this rule since the substitution process applies here to triplets of consecutive sites of coding sequences. The non-independence of the substitution process at the three coding positions (due to the specificities of the genetic code), can therefore be accounted for. Assuming that sites evolve independently does not mean that a distinct model is fitted to each site of the alignment. Estimating the parameters of these models would not make much sense in practice due to the very limited amount of phylogenetic signal conveyed by individual sites. Site independence means instead that the columns of the observed alignment were sampled randomly from the same "population of columns". The stochasticity of the substitution process running along the tree is deemed responsible to the variability of site patterns.

Some parameters of the phylogenetic model are considered to be common to all the sites in the alignment. The tree topology is typically one such parameter. The transition/transversion ratio is also generally assumed to be the same for all columns. Other parameters can vary from site to site. The rate at which substitutions accumulate is one of these parameters. Hence, different sites can have distinct rates. However, such rates are all "drawn" from the same probabilitic distribution (generally a discrete Gamma density). Hence, while different sites may have distinct rates of evolution, they all share the same *distribution* of rates.

This reasonning also applies on a larger scale. When analysing multiple genes, one can indeed assume that the same mechanism generated the different site patterns observed for every gene. Here again, we can assume that all the genes share the same underlying tree topology (commonly refered to as the "species tree"). Other parameters of the phylogenetic model, such as branch lengths for instance, might be shared across genes. However, due to the specificities of the gene evolution processes, some model parameters need to be adjusted for each gene separately. To sum up, the phylogenetic analysis of partitionned data requires flexible models with parameters, or distribution of parameters, shared across several partition elements and other parameters estimated separately for each element of the partition.

The likelihood of a data set made of the concatenation of n sequence alignments noted $D^{(1)}$, $D^{(2)}, \ldots, D^{(n)}$ is then obtained as follows:

$$\begin{aligned}\Pr(D^{(1)}, D^{(2)}, \ldots, D^{(n)}) &= \prod_{i=1}^{n} \Pr(D^{(i)}) \\ &= \prod_{i=1}^{n} \prod_{s=1}^{L_i} \Pr(D_s^{(i)}),\end{aligned}$$

where L_i is the number of site columns in partition element i. $\Pr(D_s^{(i)})$ is then obtained using Equation 1, i.e., by summing over the different classes of the mixture model that applies to site s for partition element i. Hence, the joint probability of all the partition elements is here broken down into the product of likelihood of every site for each partition element. As noted just above, any given component of the mixture model at a given particular site is shared by the other sites that belong to the same partition element and, for some of them, by sites in other partition elements (e.g., the same tree topology is shared by all the sites, throughout all the partition elements).

PhyML implements a wide variety of partition models. The only parameter that is constrained to be shared by all the partition elements is the tree topology. This constraint makes sense when considering distantly related taxa, typically inter-species data. For closely related taxa, i.e., when analysing intra-species or population-level data, not all the genes might have the same evolutionary history. Recombination events combined to the incomplete lineage sorting phenomenon can generate discrepancies between the gene trees and the underlying species tree (see [46] for a review). The phylogenetic softwares BEST [47], STEM [48] and *BEAST [49] are dedicated to the estimation of species tree phylogenies from the analysis of multi-gene data and allow gene-tree topologies to vary across genes.

Aside from the tree topology that is common to all the sites and all the partition elements, other parameters of the phylogenetic model can be either shared across partition elements or estimated separately for each of these. When analysing three partition elements, A, B and C for instance, PhyML can fit a model where the same set of branch lengths applies to A and B while C has its own estimated lengths. The same goes for the substitution model: the same GTR model, with identical parameter values, can be fitted to A and C and JC69 for instance can be used for B. The sections below give more detailed information on the range of models available and how to set up the corresponding XML configuration files to implement them.

10.3 Combining mixture and partitions in PhyML: the theory

The rationale behind mixture models as implemented in PhyML lies in (1) the definition of suitable rate matrices, equilibrium frequency vectors and relative rates of substitution and (2) the assembly of these components so as to create the classes of a mixture. The main idea behind partitionned analysis in PhyML lies in (1) the hypothesis of statistical independance of the different data partition elements and (2) distinct data partition can share model components such as rate matrices, equilibrium frequencies or distribution of rates across sites. More formally, the likelihood of a data set made of n partition elements is written as follows:

$$\begin{aligned}\Pr(D^{(1)}, D^{(2)}, \ldots, D^{(n)}) &= \prod_{i=1}^{n}\prod_{s=1}^{L_i} \Pr(D_s^{(i)}) \\ &= \prod_{i=1}^{n}\prod_{s=1}^{L_i}\sum_{c=1}^{K_i} \Pr(D_s^{(i)}|\mathcal{C}=c)\Pr(\mathcal{C}=c),\end{aligned}$$

where L_i is the number of sites in partition element i and K_i is the number of classes in the mixture model that applies to this same partition element. Each class of a mixture is made of a rate matrix M, a vector of equilibrium frequencies F and a relative rate of substitution R. Branch lengths, L and tree topology τ are also required for the calculation of the likelihood. Hence we have:

$$\begin{aligned}&\Pr(D^{(1)}, D^{(2)}, \ldots, D^{(n)}) \\ &= \prod_{i=1}^{n}\prod_{s=1}^{L_i}\sum_{c=1}^{K_i} \Pr(D_s^{(i)}|\mathcal{C}=c)\Pr(\mathcal{C}=c) \\ &= \prod_{i=1}^{n}\prod_{s=1}^{L_i}\sum_{m}^{\mathcal{M}_i}\sum_{f}^{\mathcal{F}_i}\sum_{r}^{\mathcal{R}_i} \Pr(D_s^{(i)}|M_m^{(i)}, F_f^{(i)}, R_r^{(i)}, L^{(i)}, \tau)\Pr(M_m^{(i)}, F_f^{(i)}, R_r^{(i)})\mathcal{I}(m,f,r,i)\end{aligned}$$

where $\mathcal{M}_i$, $\mathcal{F}_i$ and $\mathcal{R}_i$ are the number of rate matrices, vector of equilibrium frequencies and relative rates that apply to partition element i respectively. K_i is the number of classes in the mixture that applies to partition element i. $\mathcal{I}(m,f,r,i)$ is an indicator function that takes value 1 if the combination M_m, F_f and R_r is acually defined in the model for this particular partition element i. Its value is 0 otherwise. We therefore have $\sum_m \sum_f \sum_r \mathcal{I}(m,f,r,i) = K_i$.

In the example given in section 10.1, $\{R_1, R_2, R_3\}$ is the set of substitution rates, $\{M_1, M_2\}$ the set of rate matrices and $\{F_1, F_2\}$ the set of vectors

of equilibrium frequencies. We thus have a total of $3 \times 2 \times 2$ ways of combining the rates, matrices and frequencies, potentially making up a mixture model with 12 classes. But we define the first class of our mixture model as $\mathcal{C}_1 = \{R_1, M_1, F_1\}$, a second class as $\mathcal{C}_2 = \{R_2, M_1, F_1\}$ and the third as $\mathcal{C}_3 = \{R_3, M_2, F_2\}$. Hence, we have $\mathcal{I}(1,1,1,i)$, $\mathcal{I}(1,1,2,i)$ and $\mathcal{I}(2,2,3,i)$ equal to one while the nine other values that this indicator function takes, corresponding to the possible combinations of two vectors of frequencies, two matrices and three rates, are all zero.

As stated before, our implementation assumes that the different components of a mixture are independant. In other words, we have $\Pr(M_m^{(i)}, F_f^{(i)}, R_r^{(i)}) \propto \Pr(M_m^{(i)}) \times \Pr(F_f^{(i)}) \times \Pr(R_r^{(i)})$. In practice, the joint probability $\Pr(M_m^{(i)}, F_f^{(i)}, R_r^{(i)})$ is obtained as follows:

$$\Pr(M_m^{(i)}, F_f^{(i)}, R_r^{(i)}) = \frac{\Pr(M_m^{(i)}) \Pr(F_f^{(i)}) \Pr(R_r^{(i)}) \mathcal{I}(m,f,r,i)}{\sum_{a,b,c} \Pr(M_a^{(i)}) \Pr(F_b^{(i)}) \Pr(R_c^{(i)}) \mathcal{I}(a,b,c,i)}$$

The probabilities $\Pr(M_m^{(i)})$, $\Pr(F_f^{(i)})$ and $\Pr(R_r^{(i)})$, also called 'weights', can be fixed or estimated from the data.

10.4 The XML format and its use in PhyML

The few paragraphs below are largely inspired from the Wikipedia page that describes the XML format (http://en.wikipedia.org/wiki/XML). XML (eXtensible Markup Language) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. An XML document is divided into *markup* and *content*, which may be distinguished by the application of simple syntactic rules. Generally, strings that constitute markup either begin with the character '<' and end with a '>'. Strings of characters that are not markup are content:

XML markup and content example

```
<markup>
 content
</markup>
```

A markup construct that begins with '<' and ends with '>' is called a *tag*. Tags come in three flavors: (1) start-tags (e.g, `<section>`), end-tags (e.g., `</section>`) and empty-element tags (e.g., `<line-break />`). A *component* either begins with a start-tag and ends with a matching end-tag or consists only of an empty-element tag. The characters between the start- and end-tags, if any, are the element's content, and may contain markup, including other elements, which are called child elements.

In the following example, the element `img` has two *attributes*, `src` and `alt`: `<img src="madonna.jpg" alt="Foligno Madonna, by Raphael"/>`. Another example would be `<step number="3">Connect A to B.</step>` where the name of the attribute is "`number`" and the value is "`3`".

In practice, building a mixture model in a XML file readable by PhyML is relatively straightforward. The first step is to define the different components of each class of the mixture. Consider for instance that the fitted model will have a Gamma distribution with four classes plus a proportion of invariants. The rate component of the mixture can then be specified using the following XML code:

Γ4+I rates

```
<siterates id="SiteRates1">
  <weights  id="Distrib" family="gamma+inv" alpha=".1" \
  optimise.alpha="yes" pinv="0.4" optimise.pinv="yes">
  </weights>
  <instance id="R1" init.value="1.0"/>
  <instance id="R2" init.value="1.0"/>
  <instance id="R5" init.value="0.0"/>
  <instance id="R3" init.value="1.0"/>
  <instance id="R4" init.value="1.0"/>
</siterates>
```

In the example above, the `<siterates>` component completely defines a model of substitution rate variation across sites. This component has a particular identity, i.e., a name associated to it ("`SiteRates1`" here), which is not mandatory. This `<siterates>` component has six sub-components. The first is the `<weights>` component, followed by five `<instance>` components. The `<weights>` component defines the type of distribution that characterizes the variation of rates across sites. A discrete Gamma plus invariants is used here. Two parameters specify this distribution: the gamma shape and the proportion of invariant parameters. Their initial values are set by using the corresponding attributes and attribute values (`alpha="0.1"` and `pinv="0.4"`). Also, PhyML can optimise these parameters so as to maximise the likelihood of the whole phylogenetic model (`optimise.pinv="yes"` and `optimise.alpha="yes"`). The following five `<instance>` components define the rate classes themselves. The `id` attribute is here mandatory and must be unique to each class. Note that one of the initial (relative) rate (`init.value` attribute) is set to zero. The corresponding rate class (the third in this example) will then correspond to the invariant site category.

Having specified the part of the phylogenetic model that describes the variation of rates across sites, we can now move on to build the rest of the model. The component below defines two substitution models:

Rate matrices

```
<ratematrices id="RateMatrices">
  <instance id="M1" model="HKY85" tstv="4.0" optimise.tstv="no"/>
  <instance id="M2" model="GTR" optimise.rr="yes"/>
</ratematrices>
```

This `<ratematrices>` component sets out a list of substitution models (HKY85 and GTR here). Here again, the different elements in this list correspond to the `<instance>` sub-components. Each instance must have a unique `id` attribute for a reason that will become obvious shortly. The remaining attributes and their functions are described in Section 10.6.3.

The next "ingredient" in our phylogenetic model are vectors of nucleotide frequencies. The `<equfreqs>` component below specifies two of such vectors:

Equilibrium frequencies

```
<equfreqs id="EquFreq">
  <instance id="F1"/>
  <instance id="F2"/>
</equfreqs>

```

Now, we need to assemble these three components (rate variation across sites, rate matrices and vectors of equilibrium frequencies) into a mixture model. The `<partitionelem>` component below defines one such model:

Mixture model

```
<partitionelem id="Part1" file.name="./nucleic.txt" data.type="nt">
  <mixtureelem list="R1, R2, R3, R4, R5"/>
  <mixtureelem list="M1, M1, M1, M2, M2"/>
  <mixtureelem list="F1, F2, F1, F2, F2"/>
</partitionelem>

```

The `<partitionelem>` component defines a particular partition element. In this example, the partition element corresponds to the sequence file called `nucleic.txt`, which is an alignment of nucleotide sequences (see the `data.type` attribute value). The `<mixtureelem>` are sub-components of the `<partitionelem>` component. Each `<mixtureelem>` has a `list` attribute. Each such `list` gives the ID of components that have been defined before. For instance, the first `<mixtureelem>` refers to the five classes of the `<siterates>` component. The ordering of the different term in these list matters a lot since it is directly related to the elements in each class of the mixture model. Hence, the first element in the `<list>` attribute of the first `<mixtureelem>` added to the first element in the `<list>` attribute of the second `<mixtureelem>` plus the the first element in `<list>`

attribute of the third `<mixtureelem>` defines the first class of the mixture model. Therefore, the mixture model defined above has five classes: $\mathcal{C}_1 = \{R_1, M_1, F_1\}$, $\mathcal{C}_2 = \{R_2, M_1, F_2\}$, $\mathcal{C}_3 = \{R_3, M_1, F_1\}$, $\mathcal{C}_4 = \{R_4, M_2, F_2\}$ and $\mathcal{C}_5 = \{R_5, M_2, F_2\}$.

10.5 Setting up mixture and partition models in PhyML: the basics

Mixture models are particularly relevant to the analysis of partitionned data. Indeed, some features of evolution are gene-specific (e.g., substitution rates vary across genes). Models that can accomodate for such variation, as mixture models do, are therefore relevant in this context. However, other evolutionary features are shared across loci (e.g., genes located in the same genomic region usually have similar GC contents). As a consequence, some components of mixture models need to be estimated separately for each partition element while others should be shared by different partition elements.

Below is a simple example with a partitionned data set made of two elements, corresponding to the sequence alignment files `nucleic1.txt` and `nucleic2.txt`. Importantly, the number and names of sequences in these two alignments have to match exactly.

Two sets of branch lengths (one per partition element)

```
1
2  <branchlengths id="BranchLens">
3    <instance id="L1"/>
4    <instance id="L2"/>
5  </branchlengths>
6
7  <partitionelem id="Part1" file.name="./nucleic1.txt" data.type="nt">
8    <mixtureelem list="R1, R2, R3, R4, R5"/>
9    <mixtureelem list="L1, L1, L1, L1, L1"/>
10 </partitionelem>
11
12 <partitionelem id="Part2" file.name="./nucleic2.txt" data.type="nt">
13   <mixtureelem list="R1, R2, R3, R4, R5"/>
14   <mixtureelem list="L2, L2, L2, L2, L2"/>
15 </partitionelem>
16
```

Mixture elements with names `R1`,..., `R5` refer to the Γ4+I model defined previsouly (see Section 10.4). The `<branchlengths>` XML component defines a mixture element that had not been introduced before. It defined vectors of branch lengths that apply to the estimated phylogeny. Two instances of such vectors are defined: `L1` and `L2`. When examining the two partition elements (`<partitionelem>` component), it appears that `L1` is associated with `Part1` while `L2` is associated with `Part2`. Hence, branch lengths will be estimated separately for these two partition elements.

Note that a given partition element can only have one `branchlengths` instance associated to it. For instance, the example given below is not valid:

Invalid mixture

```
<partitionelem id="Part1" file.name="./nucleic1.txt" data.type="nt">
  <mixtureelem list="R1, R2, R3, R4, R5"/>
  <mixtureelem list="L1, L1, L1, L2, L2"/>
</partitionelem>
```

In other words, mixture of branch lengths are forbidden. One reason for this restriction is that mixture of edge lengths sometimes lead to non-identifiable models (i.e., models with distinct sets of branch lengths have the same likelihood) [50]. But mostly, combining mixture of branch lengths with mixture of rates appears like a deadly combination. Consider for instance the following model:

Invalid mixture

```
<partitionelem id="Part1" file.name="./nucleic1.txt" data.type="nt">
  <mixtureelem list="R1, R2, R3"/>
  <mixtureelem list="L1, L2, L3"/>
</partitionelem>
```

It is here impossible to tell apart branch lengths and substitution rates. Such model is strongly non-identifiable and therefore not relevant.

In the example given above, the same Γ4+I model (i.e. the same gamma shape parameter and proportion of invariant) applies to the two partition elements. It is possible to use two distinct Γ4+I models instead using the following XML code:

Two distinct Γ4+I models

```
<siterates id="SiteRates1">
  <weights  id="Distrib1" family="gamma+inv" alpha=".1" \
  optimise.alpha="yes" pinv="0.4" optimise.pinv="yes">
  </weights>
  <instance id="R1" init.value="1.0"/>
  <instance id="R2" init.value="1.0"/>
  <instance id="R5" init.value="0.0"/>
  <instance id="R3" init.value="1.0"/>
  <instance id="R4" init.value="1.0"/>
</siterates>

<siterates id="SiteRates2">
  <weights  id="Distrib2" family="gamma+inv" alpha=".1" \
  optimise.alpha="yes" pinv="0.4" optimise.pinv="yes">
  </weights>
  <instance id="R6"  init.value="1.0"/>
  <instance id="R7"  init.value="1.0"/>
  <instance id="R8"  init.value="0.0"/>
  <instance id="R9"  init.value="1.0"/>
  <instance id="R10" init.value="1.0"/>
</siterates>

<partitionelem id="Part1" file.name="./nucleic1.txt" data.type="nt">
  <mixtureelem list="R1, R2, R3, R4, R5"/>
  <mixtureelem list="L1, L1, L1, L1, L1"/>
</partitionelem>

<partitionelem id="Part2" file.name="./nucleic2.txt" data.type="nt">
  <mixtureelem list="R6, R7, R8, R9, R10"/>
  <mixtureelem list="L2, L2, L2, L2, L2"/>
</partitionelem>

```

`SiteRates1` and `SiteRates2` here define two distinct Γ4+I models. Each of these models apply to one of the two partition elements (`nucleic1.txt` and `nucleic2.txt`), allowing them to display different patterns of rate variation across sites.

10.6 XML options

10.6.1 `phyml` component

Options:

- `output.file="filename"`. The main output files of PhyML analysis will be named `filename_phyml_tree` and `filename_phyml_stats`.
- `bootstrap="nreplicates"`. Run `nreplicates` replicates for the non-parametric bootstrap analysis.
- `run.id="idstring"`. PhyML will append the string `idstring` to each output file.

- `print.json.trace="yes|true|no|false"`. PhyML will print the estimated trees, the corresponding loglikelihoods and various model parameters at multiple stages of the estimation process. This option is useful for monitoring the progress of the analysis when processing large data sets.

- `print.trace="yes|true|no|false"`. PhyML will print the estimated trees (and the corresponding loglikelihoods) at multiple stages of the estimation process. This option is also useful for monitoring the progress of the analysis when processing large data sets.

- `branch.test="aBayes|aLRT|SH|no"`. Calculate fast branch support using the aBayes method [25], aLRT [26] or SH [51] tests. These branch statistics are much faster to estimate than the bootrap proportions and usually provide good estimates of the probabilities that the corresponding edges are correctly inferred (see Anisimova et al. 2011 for more precision). By default and if no bootstrap analysis is performed, branch supports are estimated using the aBayes approach.

- `quiet="yes|no"`. Runs PhyML in quiet mode when `quiet=yes`. The program will not pause if the memory required to run the analysis exceeds 256MB and will not output the progresssion of the log-likelihood scores on the standard output.

- `memory.check="yes|no"`. By default, when processing a large data set, PhyML will pause and ask the user to confirm that she/he wants to continue with the execution of the analysis despite the large amount of memory required. Setting `memory.check=no` skips this question. It is especially useful when running PhyML in batch mode.

10.6.2 `topology` component

Each instance of the `topology` component (there should be only one...) has the following options:

- `init.tree="bionj"|"user"|"random"`. Starting tree. Default is `bionj`.

- `n.rand.starts="X"`. Number of random starting trees. Default is 5.

- `file.name="name_of_tree_file"`. In case `init.tree="user"`, this attribute is mandatory. `name_of_tree_file` is a text file containing a tree in NEWICK format.

- `optimise.topology="yes"|"true"|"no"|"false"`. The starting tree topology as defined by `init.tree` is to be optimised (or not) so as to maximise the likelihood function.

- `search="nni"|"spr"|"none"`. Tree topology search is conducted using NNI (fast), SPR (a bit slower but more accurate) or no moves.

Note: earlier versions of this manual called this attribute `optimise.tree`, which PhyML does not read: with it, the topology is always optimised.

Example of 'topology' component

```
<topology>
  <instance id="T1" init.tree="bionj" optimise.topology="true" \
   search="spr"/>
</topology>
```

10.6.3 `ratematrices` component

Each instance of a `ratematrices` component have the following options:

- `model="JC69"|"K80"|"F81"|"F84"|"HKY85"|"TN93"|"GTR"|"custom"` for nucleotide data. The default is `"HKY85"`.
  `model="LG"|"WAG"|"JTT"|"MtREV"|"Dayhoff"|"DCMut"|"RtREV"|"CpREV"|"VT"|"Blosum62"|"MtMam"|"MtArt"|"HIVw"|"HIVb"|"customaa"` for amino-acid sequences. The default is `"LG"`.

- `model.code="012345"`. For `custom` model applied to nucleotide sequences: set the string of digits that define a custom substitution model. See Table 1 on page 18 for more information about the model codes.

- `ratematrix.code="filename"`. When used in conjunction with `model="customaa"`, `filename` is the name of the file that gives the rates of substitution between amino-acids as well as their frequences at equilibrium using PAML rate matrix format. An example of such file is provided in phyml/examples/X1.mat.

- `optimise.rr="yes"|"true"|"no"|"false"`. For `custom` and `GTR` nucleotide models only: optimise the substitution rate model parameters.

- `optimise.tstv="yes"|"true"|"no"|"false"`. For `K80`, `F84`, `HKY85` and `TN93` models only: optimise the transition/transversion rate ratio.

- `tstv="value"`. For `K80`, `HKY85` and `TN93` models only: set the transition/transversion to a given value.

An instance of a `rematrices` component where a GTR or a custom model of substitutions between nucleotides is used can have a `rr` component associated to it in order to specificy the relative rates of substitutions (see example below).

Also, the `rematrices` component has the attribute `optimise.weights=yes/no` (default is `no`). If `optimise.weights=yes`, then the probabilities (or weights) or each matrix in the set of matrices defined by this component (see Equation 1), will be estimated from the data.

Example of 'rematrices' component

```
<rematrices id="RM1" optimise.weights="yes">
  <instance id="M1" model="custom" model.code="000000"/>
    <rr AC="1.2" AG="1.5" AT="2.3" CT="1.2" CG="1.5"/>
  <instance id="M2" model="GTR" optimise.rr="yes"/>
  <instance id="M3" model="WAG"/>
</rematrices>
```

10.6.4 `equfreqs` component

Each instance of a `equfreqs` component has the following options:

- `base.freqs="a,b,c,d"` where `a-d` are nucleotide frequencies. Make sure that these frequencies are separated by comas and no space character is inserted.

- `freqs="empirical|model|user|optimized"`. Character frequencies are derived from counting the number of occurence of each amino-acid in the alignment (`freqs="empirical"`), given by the substitution model (`freqs="model"`), user-defined (`freqs="user"`, only available for nucleotide data) or estimated using maximum likelihood (`freqs="optimized"`).

The `equfreqs` component has the attribute `optimise.weights=yes/no` (default is `no`). If `optimise.weights=yes`, then the probabilities (or weights) or each vector of equilibrium frequencies in the set of vectors defined by this component (see Equation 1), will be estimated from the data.

Example of 'equfreqs' component

```
<equfreqs id="EF1" optimise.weights="yes">
  <instance id="F1" base.freqs="0.25,0.25,0.25,0.25"/>
  <instance id="F2" freqs="empirical"/>
  <instance id="F3" optimise.freqs="yes"/>
</equfreqs>
```

10.6.5 `branchlengths` component

Options:

- `optimise.lens="yes"|"true"|"no"|"false"`: branch lengths are optimised or not. The default is set to `"yes"`.

Example of 'branchlengths' component

```
<branchlengths id="BL1">
  <instance id="L1" optimise.lens="yes"/>
  <instance id="L2"/>
  <instance id="L3" optimise.lens="false"/>
</branchlengths>
```

10.6.6 `siterates` component

Each instance defines a class of relative rate of substitution. It has the following options:

- `value="val"`, where `"val"` is the relative substitution rate for the corresponding class.

A `siterates` component generally includes a `weights` element that specifies the probabilitic distribution of the relative rates. The available options for such element are:

- `family="gamma|gamma+inv|freerates"`. `gamma` indicates that the distribution of the relative rates is set to be a discrete Gamma density. `gamma+inv` indicates that the relative rate model is a mixture of Gamma and invariant sites (this is the common Γ+I model). FreeRate is a model that does not use any parametric function to describe the distribution of the relative rates (see [36]). Under this option, relative rates and the corresponding frequencies of these classes are directly estimated from the data. While such approach is slightly more computationally demanding than the Γ (or Γ+I) model, it often provides a significantly better fit to the data.

- `alpha="value|optimised"`, where `value` is a real positive number. Use this option to set the gamma shape parameter to the selected value. `optimised`: the parameter is estimated from the data (see also next option).

- `optimise.alpha="yes|true|no|false"`. Optimise the shape of the Gamma distribution of relative rates (or not).

- `pinv="value|optimised"`, where `value` is in $[0, 1]$. Use this option to set the proportion of invariants to the selected value. `optimised`: the parameter is estimated from the data (see also next option).
- `optimise.pinv="yes|true|no|false"`. Optimise the proportion of invariable sites (or not).
- `optimise.freerates="yes|true|no|false"`. Optimise the parameters of the FreeRate model, i.e., the relative rates and the corresponding frequencies.

Example of 'siterates' component (discrete gamma model)

```
<siterates id="SR1">
  <instance id="R1" init.value="1.0"/>
  <instance id="R2" init.value="1.0"/>
  <instance id="R3" init.value="1.0"/>
  <instance id="R4" init.value="1.0"/>
  <weights  id="D1" family="gamma" optimise.alpha="yes" \
  optimise.pinv="no">
  </weights>
</siterates>
```

Setting up an analysis using the FreeRate mixture model of rate variation across sites [36] requires a bit more work. As in the discrete gamma model, the FreeRate model relies on definies classes of (relative) rates. In the discrete gamma model, each class has the same frequency. This is no longer the case with FreeRate. It is thus necessary to specify the frequency, or weight, of each class of rate in the XML file. The example below illustrates how these weights are defined in practice:

Example of 'siterates' component (FreeRate model)

```
<!-- Freerate model of variation of rates across sites -->
<siterates id="SR1">
  <instance id="R1" init.value="0.1"/>
  <instance id="R2" init.value="0.7"/>
  <instance id="R3" init.value="1.9"/>
  <instance id="R4" init.value="5.1"/>
  <weights  id="D1" family="freerates" optimise.freerates="yes">
    <instance appliesto="R1" value="0.4"/>
    <instance appliesto="R2" value="0.3"/>
    <instance appliesto="R3" value="0.1"/>
    <instance appliesto="R4" value="0.1"/>
  </weights>
</siterates>
```

Note that the weights do not have to sum to one. PhyML rescales them before starting the analysis such that these weights are transformed into proper probabilities. The relative rates themselves are rescaled too such that the weighted average (relative) rate is equal to one after rescaling.

10.6.7 `partitionelem` and `mixtureelem` components

Options:

- `file.name="inputfilename"`, where `inputfilename` is the name of the input sequence file (in PHYLIP format) to be analysed.
- `data.type="nt|aa"`. Specify the type of sequences to be processed (nucleotide of amino-acid sequences).
- `interleaved="yes|true|no|false"`. Interleaved (`yes|true`) or sequential format (`no|false`) for the sequence alignment.
- `optimise.tree.scale="yes|true|no|false"`. The sum of edge length (or tree size) is optimized. This option is relevant when different data partition elements point to the same set of edge lengths so that setting `optimise.tree.scale="yes"` will find the optimal ratio of tree sizes considering the two elements. In other words, the different trees all share the same (relative) edge lengths but the corresponding partition elements have different mean rates of substitution.
- `tree.scale="val"`. `val` is the value of the (relative) substitution rate. By default, its value is set to 1.0.
- `print.site.lk="yes|true|no|false"`. The likelihood at each site (and other information) will be written in a file named `inputfilename_phyml_lk`.

Each `partitionelem` element should include exactly four `mixtureelem` elements, corresponding to branch lengths, equilibrium frequencies, substitution rate model and tree topology. The ordering of in which the `mixtureelem` elements are given does not matter, though exceptions apply for the $\Gamma + I$ model (see below). The n-th element in the `list` attribute of each `mixtureelem` defines the n-th class of the mixture model. In the example given below, the first class of the mixture is made of the following elements: `T1`, `F1`, `R1` and `L1`, the second class is made of `T1`, `F1`, `R2` and `L1`, etc.

Example of 'partitionelem' component

```
1
2   <partitionelem id="partition1" file.name="./small_p1.nxs" \
3    data.type="nt" interleaved="yes">
4     <mixtureelem list="T1, T1, T1, T1"/>
5     <mixtureelem list="F1, F1, F1, F1"/>
6     <mixtureelem list="R1, R2, R3, R4"/>
7     <mixtureelem list="L1, L1, L1, L1"/>
8   </partitionelem>
9
```

In general, the ordering of the `mixtureelem` elements does not matter. However, when the model has invariable sites, then the corresponding class should be first in the list of classes provided by `mixtureelem`. For instance, in the example above, if the rates are defined as follows:

Example of 'siterates' component

```
1
2   <siterates id="SR1">
3     <instance id="R1" init.value="0.0"/>
4     <instance id="R2" init.value="1.0"/>
5     <instance id="R3" init.value="1.0"/>
6     <instance id="R4" init.value="1.0"/>
7     <weights  id="D1" family="gamma+inv" optimise.alpha="yes" \
8     optimise.pinv="no">
9     </weights>
10  </siterates>
```

then `R1` corresponds to the invariable rate class (as `init.value="0.0"`). As `R1` is first in the `mixtureelem` (see line 6 in the example of `'partionelem'` given above), PhyML will print out an explicit error message and bail out. One way to avoid this shortcoming is to define `mixtureelem` as `R4, R2, R3, R1` instead.

10.7 Example: GTR + Γ4 + I

The example below provides all the required options to fit a Γ4+I model to a single alignment of nucleotide sequences under the GTR model of substitution using a SPR search for the best tree topology. The `phyml` component sets the name for the analysis to `simple.example`, meaning that each output file will display this particular string of characters. Also, the tree and statistics file names will begin with `p1.output`. The tree topology will be estimated so as to maximise the likelihood and the topology search algorithm used here is SPR, as indicated by the value of the corresponding attribute (i.e., `search="spr"`). Only one vector of branch lengths will be used here since only one partition element will be processed. Hence, the `<branchlengths>` component only has one `<instance>` sub-component. Also, a single GTR model will apply to all the classes for the mixture model – the `<ratematrices>` component has only one `<instance>` sub-component, corresponding to this particular substitution model. The next component, `<equfreqs>`, indicates that a single vector of equilibrium frequencies will apply here. Next, the `<siterates>` component has five `<instance>` sub-components. Four of these correspond to the non-zero relative rates of evolution a defined by a discrete Gamma distribution. The last one (`<instance id="R5" value="0.0"/>`) defines the class of the mixture corresponding to invariable sites. The `<weight>` component indicates that a Γ+I model will

be fitted here. The shape parameter of the Gamma distribution and the proportion of invariants will be estimated from the data. The `<partitionelem>` gives information about the sequence alignment (the corresponding file name, the type of data and the alignment format). The `<mixtureelem>` components next define the mixture model. Each class of the fitted model corresponds to one column, with the first column made of the following elements: `T1, M1, F1, R1` and `L1`. The second class of the mixture is made of `T1, M1, F1, R2, L1` and so forth.

Simple PhyML XML example

```
<phyml runid="simple.example" output.file="p1.output">

  <topology>
    <instance id="T1" init.tree="bionj" optimise.topology="yes" \
    search="spr"/>
  </topology>

  <branchlengths id="BL1">
    <instance id="L1" optimise.lens="yes"/>
  </branchlengths>

  <ratematrices id="RM1">
    <instance id="M1" model="GTR"/>
  </ratematrices>

  <equfreqs id="EF1">
    <instance id="F1"/>
  </equfreqs>

  <siterates id="SR1">
    <instance id="R1" value="1.0"/>
    <instance id="R2" value="1.0"/>
    <instance id="R3" value="1.0"/>
    <instance id="R4" value="1.0"/>
    <instance id="R5" value="0.0"/>
    <weights  id="D1" family="gamma+inv" optimise.alpha="yes" \
    optimise.pinv="yes">
    </weights>
  </siterates>

  <partitionelem id="partition_elem1" file.name=\
  "./p1.seq" data.type="nt" interleaved="yes">
    <mixtureelem list="T1, T1, T1, T1, T1"/>
    <mixtureelem list="M1, M1, M1, M1, M1"/>
    <mixtureelem list="F1, F1, F1, F1, F1"/>
    <mixtureelem list="R1, R2, R3, R4, R5"/>
    <mixtureelem list="L1, L1, L1, L1, L1"/>
  </partitionelem>

</phyml>

```

10.8 Example: LG4X

LG4X and LG4M are now available directly with `-m LG4X` and `-m LG4M` (Section 9.1); the XML description below remains useful to modify these mod-

els. The example below shows how to fit the LG4X model [33] to a given alignment of amino-acid sequences (file `M587.nex.Phy`). LG4X is a mixture model with four classes. Each class has its own rate and corresponding frequencies (hence the use of the FreeRate model below, see the `<siterates>` component). In the particular example given here, the rate values and frequencies are set by the users. These parameters will then be optimized by PhyML (`optimise.freerates="yes"`). Each class also has its own rate matrix and vector of equilibrium frequencies, which need to be provided by the user (Note that these matrices can be downloaded from the following web address: http://www.atgc-montpellier.fr/download/datasets/models/lg4x/LG4X_4M.txt. They are also provided in the PhyML package `example/lg4x/` directory.)

LG4X

```
<phyml run.id="lg4x" output.file="M587.tests" branch.test="no">

  <!-- Tree topology: start with BioNJ and then SPRs -->
  <topology>
    <instance id="T1" init.tree="user" file.name="user_tree.txt" \
    search="spr" optimise.topology="no"/>
  </topology>


  <!-- Four rate matrices, read from files -->
  <ratematrices id="RM1">
    <instance id="M1" model="customaa" ratematrix.file="X1.mat"/>
    <instance id="M2" model="customaa" ratematrix.file="X2.mat"/>
    <instance id="M3" model="customaa" ratematrix.file="X3.mat"/>
    <instance id="M4" model="customaa" ratematrix.file="X4.mat"/>
  </ratematrices>

  <!-- Freerate model of variation of rates across sites -->
  <siterates id="SR1">
    <instance id="R1" init.value="0.197063"/>
    <instance id="R2" init.value="0.750275"/>
    <instance id="R3" init.value="1.951569"/>
    <instance id="R4" init.value="5.161586"/>
    <weights  id="D1" family="freerates" optimise.freerates="yes">
      <instance appliesto="R1" value="0.422481"/>
      <instance appliesto="R2" value="0.336848"/>
      <instance appliesto="R3" value="0.180132"/>
      <instance appliesto="R4" value="0.060539"/>
    </weights>
  </siterates>

  <!-- Amino-acid equilibrium freqs. are given by the models -->
  <equfreqs id="EF1">
    <instance id="F1" freqs="model"/>
    <instance id="F2" freqs="model"/>
    <instance id="F3" freqs="model"/>
    <instance id="F4" freqs="model"/>
  </equfreqs>


  <!-- One vector of branch lengths -->
  <branchlengths id="BL1" >
    <instance id="L1" optimise.lens="yes"/>
  </branchlengths>


  <!-- Mixture model assemblage -->
  <partitionelem id="partition1" file.name="M587.nex.Phy" \
  data.type="aa" interleaved="yes">
    <mixtureelem list="T1, T1, T1, T1"/>
    <mixtureelem list="M1, M2, M3, M4"/>
    <mixtureelem list="F1, F2, F3, F4"/>
    <mixtureelem list="R1, R2, R3, R4"/>
    <mixtureelem list="L1, L1, L1, L1"/>
  </partitionelem>

</phyml>
```

In order to fit the LG4X model to the `proteic` sequence file provided in the `examples/` directory, simply type `./phyml --xml=../examples/lg4x/lg4x.xml` (assuming the PhyML binary is in-

stalled in the `src/` directory). You can of course slightly tweak the file `../examples/lg4x/lg4x.xml` and use it as a template to fit this model to another data set.

10.9 Example: CAT-like model

The CAT model [52] is a mixture model whereby each site may have its own exchangeability rate matrix and vector of state frequencies. In its original exposition, the number of classes in the mixture was also a parameter to be estimated from the data. PhyML uses a fixed number of classes instead. This lesser degree of sophistication facilitates the interpretation of parameter estimates. Examination of the statistics and site-likelihood files produced by PhyML under the CAT-like model makes it indeed straightforward to spot sites with peculiar substitution pattern (e.g., sites that sustained only certain types of substitutions, for instance between purines or between pyrimidines).

In the example that follows, the CAT-like model implemented has six rate classes goverened by a FreeRate model. The first and second class share the same GTR matrix (M1) and vector of nucleotide frequencies (F1) (see the two leftmost columns in the assemblage matrix). The third and fourth also share the same GTR matrix (M2) and vector of frequencies (F2) which are estimated independantly from M1 and F1. Finally, the last two classes of the mixture share M3 and F3, which are independant from the other rate matrices and state frequencies.

CAT

```
<phyml runid="CAT" output.file="example" branch.test="no" \
print.json.trace="no" print.site.lk="yes">

<topology>
<instance id="T1" init.tree="bionj"  optimise.topology="yes" \
search="spr"/>
</topology>

<branchlengths id="BL1">
  <instance id="L1" optimise.lens="yes"/>
</branchlengths>

<!-- GTR rate matrices -->
<ratematrices id="RM1">
  <instance id="M1" model="GTR" optimise.rr="yes"/>
  <instance id="M2" model="GTR" optimise.rr="yes"/>
  <instance id="M3" model="GTR" optimise.rr="yes"/>
</ratematrices>


<!-- Vectors of nucleotide frequencies -->
<equfreqs id="EF1">
  <instance id="F1" optimise.freqs="yes"/>
  <instance id="F2" optimise.freqs="yes"/>
  <instance id="F3" optimise.freqs="yes"/>
</equfreqs>


<!-- Variability of rates across sites -->
<siterates id="SR1">
  <instance id="R1" init.value="1.0"/>
  <instance id="R2" init.value="1.0"/>
  <instance id="R3" init.value="1.0"/>
  <instance id="R4" init.value="1.0"/>
  <instance id="R5" init.value="1.0"/>
  <instance id="R6" init.value="1.0"/>
  <weights id="D1" family="freerates" optimise.alpha="yes" \
  optimise.pinv="yes">
  </weights>
</siterates>


<!-- Assemblage -->
<partitionelem id="partition1" file.name="./seqfile.phy" \
data.type="nt" interleaved="no" optimise.tree.scale="no">
  <mixtureelem list="T1,T1,T1,T1,T1,T1"/>
  <mixtureelem list="R1,R2,R3,R4,R5,R6"/>
  <mixtureelem list="M1,M1,M2,M2,M3,M3"/>
  <mixtureelem list="F1,F1,F2,F2,F3,F3"/>
  <mixtureelem list="L1,L1,L1,L1,L1,L1"/>
</partitionelem>


</phyml>
```

10.10 Example: multiple partition elements

The example below gives the complete XML file to specify the analysis of three partition elements, corresponding to the nucleotide sequence files

small_p1_pos1.seq, small_p1_pos2.seq and small_p1_pos3.seq in interleaved PHYLIP format. Importantly, the number and names of sequences in these three alignments match exactly. small_p1_pos1.seq is fitted with the HKY85 model of substitution (with the transition/transversion ratio being estimated from the data), combined to a $\Gamma 4$ model of rate variation across sites (with the gamma shape parameter being estimated from the data). small_p1_pos2.seq is fitted to a custom substitution model with the constraint $A \leftrightarrow G{=}C \leftrightarrow T$. The nucleotide frequencies are set to $\frac{1}{4}$ here. The model does not allow substitution rates to vary across sites. small_p1_pos3.seq is fitted using a GTR model conbined to a $\Gamma 4{+}\mathrm{I}$ model of rate variation across sites. Note that the equilibrium nucleotide frequencies for the fourth and fifth class of the mixture are set to be equal to that estimated from the first partition element (i.e., F1) . The initial phylogeny is built using BioNJ and the tree topology is to be estimated using a NNI search algorithm.

Example of PhyML XML file

```
<phyml runid="nnisearch" output.file="small_p1_output">

  <topology>
    <instance id="T1" init.tree="bionj" optimise.topology="yes" \
    search="nni"/>
  </topology>

  <branchlengths id="BL1">
    <instance id="L1" optimise.lens="yes"/>
    <instance id="L2"/>
    <instance id="L3"/>
  </branchlengths>

  <rematrices id="RM1">
    <instance id="M1" model="HKY85" optimise.tstv="yes"/>
    <instance id="M2" model="custom" model.code="102304" \
    optimise.rr="yes"/>
    <instance id="M3" model="GTR"/>
  </rematrices>

  <equfreqs id="EF1">
    <instance id="F1"/>
    <instance id="F2" base.freqs="0.25,0.25,0.25,0.25"/>
    <instance id="F3"/>
  </equfreqs>

  <siterates id="SR1">
    <instance id="R1" value="1.0"/>
    <instance id="R2" value="1.0"/>
    <instance id="R3" value="1.0"/>
    <instance id="R4" value="1.0"/>
    <weights  id="D1" family="gamma" optimise.alpha="yes" \
    optimise.pinv="no">
    </weights>
  </siterates>

  <siterates id="SR2">
    <instance id="R8" value="1.0"/>
    <weights  id="D2" family="gamma" optimise.alpha="yes" \
    optimise.pinv="yes">
    </weights>
  </siterates>

  <siterates id="SR3">
    <instance id="R10" value="1.0"/>
    <instance id="R11" value="1.0"/>
    <instance id="R12" value="1.0"/>
    <instance id="R13" value="1.0"/>
    <instance id="R14" value="1.0"/>
    <weights  id="D3" family="gamma" optimise.alpha="yes" \
    optimise.pinv="yes">
    </weights>
  </siterates>

```

Example of PhyML XML file (ctnd)

```
  <partitionelem id="partition_elem1" file.name=\
  "./small_p1_pos1.seq" data.type="nt" interleaved="yes">
    <mixtureelem list="T1, T1, T1, T1"/>
    <mixtureelem list="M1, M1, M1, M1"/>
    <mixtureelem list="F1, F1, F1, F1"/>
    <mixtureelem list="R1, R2, R3, R4"/>
    <mixtureelem list="L1, L1, L1, L1"/>
  </partitionelem>

  <partitionelem id="partition_elem2" file.name=\
  "./small_p1_pos2.seq" data.type="nt" interleaved="yes">
    <mixtureelem list="T1"/>
    <mixtureelem list="M2"/>
    <mixtureelem list="R8"/>
    <mixtureelem list="F2"/>
    <mixtureelem list="L2"/>
  </partitionelem>

  <partitionelem id="partition_elem3" file.name=\
  "./small_p1_pos3.seq" data.type="nt" interleaved="yes">
    <mixtureelem list="T1, T1, T1, T1, T1"/>
    <mixtureelem list="M3, M3, M3, M3, M3"/>
    <mixtureelem list="R10, R11, R12, R13, R14"/>
    <mixtureelem list="F3, F3, F3, F1, F1"/>
    <mixtureelem list="L3, L3, L3, L3, L3"/>
  </partitionelem>

</phyml>

```

10.11 Branch lengths with invariants and partionned data

Accommodating for models with invariable sites applying to some elements of a partitioned data, with these elements sharing the same set of edge lengths can lead to inconsistencies. Consider for instance a partitioned data set with two elements. Assume that these two elements share the same set of edge lengths. Also, consider that GTR+I applies to the first element and HKY applies to the second. Now, the expected number of substitutions per site for the first element of the partition is equal to $(1-p)l$, where p is the estimated proportion of invariants and l is the maximum-likelihood estimate for the length of that specific edge. For the second element of the partition, the expected number of substitutions per site is equal to l, rather than $(1-p)l$. While l are common to the two elements, matching the specification of the input model, the actual edge lengths do differ across the two partition elements. Please be aware that, due to the programming structure implemented in PhyML, the program will only return one value here, which will be equal to $(1-p)l$.

11 Citing PhyML

The "default citation" for PhyML is:

- "New algorithms and methods to estimate maximum-likelihood phylogenies: assessing the performance of PhyML 3.0". Guindon S., Dufayard J.F., Lefort V., Anisimova M., Hordijk W., Gascuel O. 2010, *Systematic Biology*, 59(3):307-321

The "historic citation" for PhyML is:

- "A simple, fast and accurate algorithm to estimate large phylogenies by maximum likelihood" Guindon S., Gascuel O. 2003, *Systematic Biology*, 52(5):696-704

12 Other programs

PhyML is software package that provides tools to tackle problems other than estimating maximum likelihood phylogenies. Installing these tools and processing data sets is explained is the following sections.

12.1 PhyTime

PhyTime is a program that uses Bayesian sampling techniques to infer phylogenies from the analysis of genetic and fossil data. Edge lengths (and node heights) are expressed in calendar time units as opposed to expected number of substitutions as in PhyML. PhyTime can thus be used to date past evolutionary events. The main original features compared to other software for molecular dating are:

- a model of rate evolution whereby rates can vary along each branch (other software consider that the rates of evolution vary from one branch to another but stay constant along any given branch) [37].
- the opportunity to accomodate for uncertainty in calibration information. A single time constraint can apply to multiple clades with corresponding probabilities as determined from prior analysis of fossil data [53].

12.1.1 Installing PhyTime

Compiling PhyTime is straightforward on Unix-like machines (i.e., linux and MacOS systems). PhyTime is not readily available for Windows machines but compilation should be easy on this system too. In the 'phyml' directory, where the 'src/' and 'doc/' directories stand, enter the following commands:

```
./configure --enable-phytime;
make clean;
make;
```

This set of commands generates a binary file called `phytime` which can be found in the 'src/' directory.

12.1.2 Running PhyTime

Passing options and running PhyTime on your data set is quite similar to running PhyML using an XML parameter input file, i.e., a typical run would be launched using the following command: `./phytime --xml=./dating_example.xml`, assuming that the `phytime` binary file is in the same directory as the XML control file `dating_example.xml`, which happens to be the current directory here. The main differences between PhyML and PhyTime are explained below:

- Unlike PhyML, PhyTime requires calibration (along with genetic sequence) data as input. The format for defining the relevant time intervals is described below.
- PhyTime does not allow partitionned (i.e., multigene) analysis yet.

12.1.3 PhyTime input

As stated above, PhyTime takes as input an XML file very similar to those compatible with PhyML. An example is given thereafter. The first part of the input file (up to the `partitionelem` block) corresponds to a standard XML input file for PhyML. Note however that the tag name "`phytime`" replaces "`phyml`". The second part is PhyTime-specific. Three new tags can be found here: `lineagerates`, `clade` and `calibration`.

The `lineagerates` element is used to define the model of rate variation across lineages. Its only attribute is `model` which can take the following values:

- `model=strictclock` or `model=clock` implements the strict clock model, i.e., all lineages evolve at the same instantaneous rate, at any point in time.

- `model=lognormal` or `model=normal`. The logarithm of the average rate along each branch is distributed as a normal distribution. The mean and variance of this distribution are the same for all branches. More precisely, let R_b be the average rate of substitution along edge b. We have $R_b = X_b \cdot \mu$, where μ is the "baseline" (or average over lineages) rate of substitution and X_b is thus the branch-specific *relative* rate of substitution. The lognormal model has $\log(X_b) \sim \mathcal{N}(1, \nu)$, i.e., the logarithm of the relative rate is normally distributed with mean (mode and median) set to 1.0 and standard deviation equal to ν.

- `model=geometricbrownian` or `model=brownian` or `model=geo`. The logarithm of the rate of substitution evolves according to a Brownian motion process along the tree. More precisely, the logarithm of the rate of evolution at the end of edge b, noted as $Y_b^{\text{stop}} = \log(R_b^{\text{stop}})$, is normally distributed with variance $v := \nu \cdot \Delta t_b$, where Δt_b is the (calendar) time elapsed along edge b, and mean equal to $Y_b^{\text{start}} - \frac{1}{2}v$, so that $\mathbb{E}(R_b^{\text{stop}}) = \mathbb{E}(R_b^{\text{start}})$. Under the geometric Brownian motion model, the average rate of evolution is also random. This variability is taken into account in the calculation of the transition between nucleotides or amino-acids along edges. See Guindon, 2012, *Syst. Biol.* for more information.

The `clade` element is used to define a subset of taxa. Each of these taxa is given in a `taxon` element. The value that each taxon takes is a string corresponding to the name of one of the sequences in the alignment file. In the example that follows, `Gymno_Araucaria`, `Gymno_Ginko`, `Gymno_Juniperus` and `Gymno_Juniperus` are three taxa that define a clade called `Gymno1`. Note that this clade may not be monophyletic. In fact, it is not considered as such (i.e., monophyletic) during the inference. The second important element is `calibration`. It defines time intervals corresponding to the time of diversification (i.e., the time of the most recent common ancestor) of the set of taxa the calibration points to. In other words, a given time interval defines the calibration for the timing of the crown node of a given clade. Each interval is defined using the `upper` and `lower` tags. The upper (resp. lower) bound for each interval is a date expressed in any unit you fancy.

Example of PhyTime XML file

```
<phytime run.id="June2018" output.file="out" \
mcmc.chain.len="1E+7" mcmc.sample.every="100" \
mcmc.print.every="50" mcmc.burnin="10000" \
mutmap="no">

<!-- Tree topology -->
<topology>
  <instance id="T1" init.tree="bionj" optimise.tree="yes"/>
</topology>


<!-- Model of rate variation across lineages -->
<lineagerates model="lognormal"/>

<!-- Substitution model -->
<ratematrices id="RM1">
  <instance id="M1" model="HKY85" optimise.tstv="no" tstv="4.0"/>
</ratematrices>


<!-- Freerate model of variation of rates across sites -->
<siterates id="SR1">
  <instance id="R3" init.value="0.5"/>
  <instance id="R2" init.value="1.0"/>
  <instance id="R1" init.value="2.0"/>
  <weights  id="D1" family="freerates" optimise.freerates="no">
    <instance appliesto="R3" value="0.33"/>
    <instance appliesto="R2" value="0.33"/>
    <instance appliesto="R1" value="0.33"/>
  </weights>
</siterates>

<!-- Nucleotide frequencies -->
<equfreqs id="EF1">
  <instance id="F1" optimise.freqs="no"/>
</equfreqs>


<!-- Vector of edge lengths -->
<branchlengths id="BL1" >
  <instance id="L1" optimise.lens="no"/>
</branchlengths>

<!-- Model assembly -->
<partitionelem id="partition1" file.name="./seq.txt" \
data.type="nt" interleaved="no">
  <mixtureelem list="T1, T1, T1"/>
  <mixtureelem list="M1, M1, M1"/>
  <mixtureelem list="F1, F1, F1"/>
  <mixtureelem list="R1, R2, R3"/>
  <mixtureelem list="L1, L1, L1"/>
</partitionelem>
```

Example of PhyTime XML file

```
  <clade id="Gymno1">
    <taxon value="Gymno_Araucaria"/>
    <taxon value="Gymno_Ginkgo"/>
    <taxon value="Gymno_Juniperus"/>
    <taxon value="Gymno_Sciadopitys"/>
  </clade>

  <clade id="Gymno2">
    <taxon value="Gymno_Araucaria"/>
    <taxon value="Gymno_Juniperus"/>
    <taxon value="Gymno_Sciadopitys"/>
  </clade>

  <calibration id="cal1">
    <lower>40</lower>
    <upper>60</upper>
    <appliesto clade.id="Gymno1" probability="0.8"/>
    <appliesto clade.id="Gymno2" probability="0.2"/>
  </calibration>

  <calibration id="cal2">
    <lower>50</lower>
    <upper>200</upper>
    <appliesto clade.id="Gymno1">
    </appliesto>
  </calibration>

</phytime>
```

12.1.4 Accounting for calibration uncertainty

It is not always obvious to determine with full precision and accuracy where in the tree a given fossil branches in *a priori*. For instance, a first morphological feature of an ancestral (fossilized) species might be shared by species A but not B nor C, while a second feature might be displayed by A and B but not C. Assuming for simplicity that the true tree topology is ((A,B),C), then the first feature calibrates the most recent common ancestor (MRCA) of A and B. Indeed, this ancestor cannot be younger than the fossil itself because, if this was the case, then A *and* B would display the morphological feature of interest. In a similar fashion, the second feature suggests that the calibration should instead apply to the MRCA of A, B and C. Therefore, assuming that the fossil was discovered in a well-defined geological layer which age range is known without ambiguity, there is still uncertainty around the node in the tree this time interval calibrates. In the particular example given above, it is not clear whether the calibration constraints applies to the node corresponding to the MRCA of A and B, or the root of the tree.

PhyTime accomodates for this uncertainty by giving a probability to the two scenari: with probability α the calibration time interval applies to the MRCA of species A and B and with probability $1-\alpha$, this interval calibrates

the age of the MRCA of A, B and C.

It is fairly straightforward to set up an analysis that incorporates probabilistic distributions on calibrations in PhyTime. The section of the XML file below gives a simple example whereby the calibration "`cal1`" applies to clade "`Gymno1`" (i.e., the smallest clade in the tree that displays the four corresponding taxa) with probability 0.8 and to clade "`Gymno2`" with probability 0.2. Also, calibration `cal2` applies to "`Gymno1`" with probability 1.0, which is implicit here and thus does not need to be specified in the XML file.

Calibrating with uncertainty

```
  <clade id="Gymno1">
    <taxon value="Gymno_Araucaria"/>
    <taxon value="Gymno_Ginkgo"/>
    <taxon value="Gymno_Juniperus"/>
    <taxon value="Gymno_Sciadopitys"/>
  </clade>

  <clade id="Gymno2">
    <taxon value="Gymno_Araucaria"/>
    <taxon value="Gymno_Juniperus"/>
    <taxon value="Gymno_Sciadopitys"/>
  </clade>

  <calibration id="cal1">
    <lower>40</lower>
    <upper>60</upper>
    <appliesto clade.id="Gymno1" probability="0.8"/>
    <appliesto clade.id="Gymno2" probability="0.2"/>
  </calibration>

  <calibration id="cal2">
    <lower>50</lower>
    <upper>200</upper>
    <appliesto clade.id="Gymno1">
    </appliesto>
  </calibration>

</phytime>
```

Note that in this example, it can be the case that both `cal1` and `cal2` apply to the same clade (`Gymno1`). In that particular situation, the actual calibration interval is conservative and derives from the overlap of the two calibration time intervals (one corresponding to `cal1`, the other to `cal2`). The MRCA of `Gymno_Araucaria`, `Gymno_Ginko`, `Gymno_Juniperus` and `Gymno_Juniperus` has an age that falls here in the $[50, 60]$ interval.

12.1.5 MCMC settings

PhyTime estimates the joint posterior distribution of the phylogenetic model parameters using an MCMC algorithm. This algorithm relies on sampling values of these parameters iteratively. The default number of iterations is

fixed to 1E+07 and values of parameters are recorded every 1E+03 iteration of the MCMC algorithm. The burn-in period, during which tuning parameters of the MCMC as adjusted, lasts for 1E+04 iterations. These three parameters can be modified by setting the values of the attributes `mcmc.chain.len`, `mcmc.sample.every` and `mcmc.burnin` in the XML parameter file accordingly. For instance, the first line of this file could look as follows:

MCMC settings

```
<phytime
run.id="example" output.file="small" mcmc.chain.len="1E+7"
mcmc.sample.every="100" mcmc.print.every="50" mcmc.burnin="10000">
```

The majority of analyses will not warrant 1E+07 iterations to converge to the target distribution (i.e., converge to the solution). I thus recommend monitoring each analysis by loading on a regular basis the statistics output file produced by PhyTime and interrupt the analysis once the effective sample sizes of all parameters have reached 200.

The MCMC algorithm implemented in PhyTime relies on multiple operators that update different parameters, or the same parameter in different ways. The weights of these operators, i.e., the frequency with which they are applied, were adjusted for the analysis of the plant data set described in [53]. If you think these default values are not suitable for your own analysis, please feel free to have a look at the function `MCMC_Complete_MCMC` in the file `mcmc.c` to make the appropriate changes to the values of `mcmc->move_weight[XXX]`, where `XXX` is the name of the operator. Alternatively, you might want to send me an email (guindon@lirmm.fr) if you are unsure about all this.

It is generally very useful to compare the posterior estimates of model parameters to that obtained from a sampler that ignores sequence data (i.e., the Bayesian sampler only takes as input data the calibration information). Such analysis helps quantifying how sensitive are the marginal posterior estimates to the estimates one would get prior collecting and analysing genetic sequences. The attribute `ignore.sequences` used in the `phytime` tag permits to run such analysis. The first lines of a corresponding XML file would then look as follows:

Sampling from the prior in PhyTime

```
<phytime run.id="prior" output.file="seq" ignore.sequences>
```

12.1.6 PhyTime output

The program PhyTime generates two output files. The statistics file is called '`outputfile_phytime_stats_runid.txt`'. It lists the node times and

(relative) substitution rates on edges sampled during the estimation process. It also gives the sampled values for other parameters, such as the autocorrelation of rates (parameter 'nu'), and the rate of evolution (parameter 'clock') amongst others. This output file can be analysed with the program Tracer from the BEAST package (http://beast.bio.ed.ac.uk/Main_Page) or Icylog (http://tgvaughan.github.io/icylog/). It is also possible to process this file using R here. Section 12.1.7 of this document gives more information about the statistics output file. The second file is called '`outputfile_phytime_trees_runid.txt`'. It is the list of rooted trees that were collected during the estimation process, i.e., phylogenies sampled from the posterior density of trees. This file can be processed using the software TreeAnnotator, also part of the BEAST package (see http://beast.bio.ed.ac.uk/Main_Page) in order to generate confidence sets for the node time estimates.

12.1.7 An example of PhyTime input and output files

The directory `phyml/examples/phytime/` provides a sequence alignment (`seq.txt`) and an XML input file (`dating_example.xml`) that can be used to run an analysis (using the following command: `phytime --xml=dating_example.xml`). The XML file is very similar to that described above and will thus not be discussed further. The columns of the statistics output file are as follows:

- `sample`: the index of the iteration in the MCMC algorithm. For instance, a sample value of 100 means that 100 "moves" in the MCMC have been applied so far, one such move can correspond to an attempt to modify the tree topology, or height of some internal nodes, etc.
- `lnL(posterior)`: the logarithm of the posterior density of the current model parameters. It is this function that the MCMC attempts to get samples from.
- `lnL(seq)`: the logarithm of the conditional probability of the sequence alignment given the current model parameters.
- `lnL(times)`: the logarithm of the probability density of the phylogeny evaluated under the birth and death branching process.
- `lnL(rates)`: the logarithm of the probability density of the relative substitution rates along every edge of the current phylogeny.

- `birth`: sampled values of the birth parameter. Histogram of these values provide an estimate of the marginal posterior distribution for this parameter.
- `death`: death parameter for the birth-death branching process.
- `clock`: mean rate of substitution.
- `root`: age of the root node, i.e. the MRCA of all taxa in the sample.
- `tstv`: transition/transversion ratio (the substitution model used here is HKY, as selected in the `param.xml` file).
- `nu`: rate autocorrelation parameter.
- `rr0`, `rr1`, `rr2`, `pr0`, `pr1`, `pr2`: relative rate of substitution and the corresponding frequencies. The mixture model of rate variation used in this analysis is the FreeRate model [36] with three rate classes. These columns give the values of these six parameters (though there are only four free parameters) sampled from the target (posterior) distribution.
- `t(calib:cal1_clade:Gymno1)`: sampled age of the MRCA of `Gymno1` when calibration `cal1` applies to this particular node.
- `t(calib:cal1_clade:Gymno2)`: sampled age of the MRCA of `Gymno2` when calibration `cal1` applies to this node.
- `t(calib:cal2_clade:Gymno1)`: sampled age of the MRCA of `Gymno1` when calibration `cal12` applies to this node.
- `clade(calib:cal1)`: the clade id (`Gymno1` or `Gymno2`) calibration `cal1` applies to (0 for `Gymno1` and 1 for `Gymno2`).
- `clade(calib:cal2)`: the clade id calibration `cal2` applies to.
- `br0`, ..., `br41`: relative rate on all edges in the tree.
- `t22`, ..., `t42`: ages of all internal nodes.
- The remaining columns provides information about the acceptance rate of various operators in the MCMC and the corresponding tuning parameters.

The script below can be copied and pasted into R in order to produce the plots in Figure 4 (assuming R was launched from the `phyml/examples/phytime` directory). The two plots at the top left and center give the trace of the node age of the MRCA of `Gymno1` when calibration `cal1` applies to it (left) and that of `Gymno2` when the same calibration `cal1` applies to it (centre). The plot on the top right gives the clade (which of `Gymno1` or `Gymno2`) the calibration `cal1` applies to.

The analysis of these plots shows the impact of applying the calibration constraint `cal1` to `Gymno1` and `Gymno2` alternatively. When `cal1` applies to `Gymno2` (i.e., (*Gymno Araucaria*, *Gymno Juniperus*, *Gymno Sciadopitys*)) the age of the MRCA of `Gymno1` ((i.e., (*Gymno_Ginkgo*, *Gymno Araucaria*, *Gymno Juniperus*, *Gymno Sciadopitys*) is free to wander towards older values compared to the situation where both `cal1` and `cal2` apply to `Gymno1`. The age of the MRCA of `Gymno2` is here bound to fall in the $[40, 60]$ time interval. When `cal1` applies to `Gymno1` instead, then the MRCA of that clade has to fall in the $[50, 60]$ interval, while that of `Gymno2` has to be younger than the MRCA of `Gymno1`.

R script to produce traces from the PhyTime statistics file

```
file=paste("out_phytime_stats_June2018.txt",sep="");
d=read.table(file,header=T);
idx=floor(0.1*length(d$sample)):length(d$sample)
par(mfrow=c(2,3),mar=c(5,4,2,2));
plot(d$sample[idx],d$t.calib.cal1_clade.Gymno1.[idx],type="l",
col="red",ylab="Time",xlab="Sample",main="Time Gymno1")
plot(d$sample[idx],d$t.calib.cal1_clade.Gymno2.[idx],type="l",
col="red",ylab="Time",xlab="Sample",main="Time Gymno2")
plot(d$sample[idx],d$clade.calib.cal1.[idx],type="l",col="black",
ylab="Clade",xlab="Sample",main="(0->Gymno1, 1->Gymno2)")
plot(d$sample[idx],d$lnL.seq.[idx],type="l",col="orange",
ylab="log(Probability)",xlab="Sample",main="Likelihood sequences")
plot(d$sample[idx],d$lnL.posterior.[idx],type="l",col="orange",
ylab="log(density)",xlab="Sample",main="Joint posterior")
plot(d$sample[idx],d$lnL.times.[idx],type="l",col="orange",
ylab="log(density)",xlab="Sample",main="Branching process")
```

12.1.8 Citing PhyTime

The "default citation" is:

- Guindon S. "From trajectories to averages: an improved description of the heterogeneity of substitution rates along lineages", *Systematic Biology*, 2013. 62(1): 22-34.

If you are using mutliple calibrations on a given clade, please cite the following article:

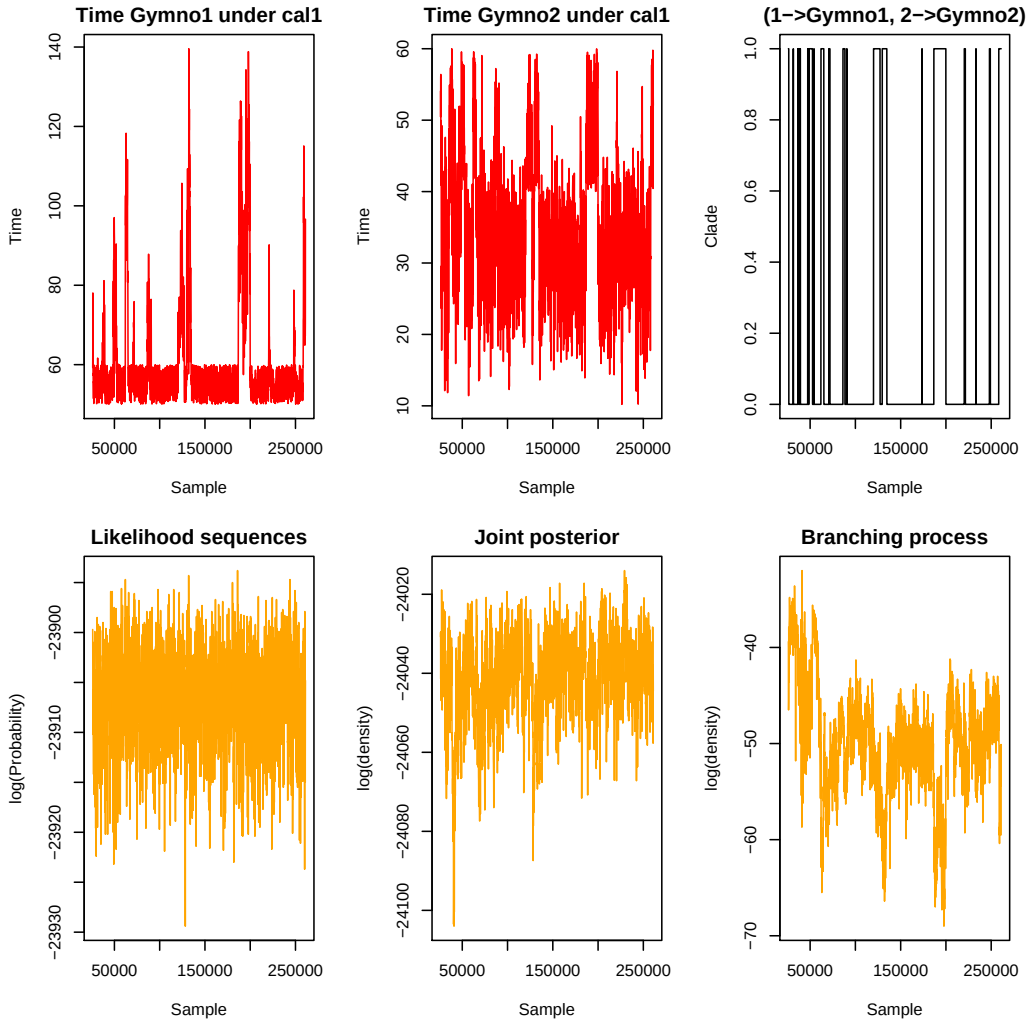


Figure 4. **Traces from the statistics output file produced by PhyTime.**

- Guindon S. "Accounting for calibration uncertainty: Bayesian molecular dating as a "doubly intractable" problem", *Systematic Biology*, 2018. 67(4): 651-661.

An earlier article also describes some of the methods implemented in PhyTime:

- Guindon S. "Bayesian estimation of divergence times from large data sets", *Molecular Biology and Evolution*, 2010, 27(8):1768:81.

12.2 PhyloGeo

PhyloGeo is a program that implements the competition-dispersal phylogeography model described in Ranjard, Welch, Paturel and Guindon "Modelling competition and dispersal in a statistical phylogeographic framework". Accepted for publication in *Systematic Biology*.

It implements a Markov Chain Monte Carlo approach that samples from the posterior distribution of the three parameters of interest in this model, namely the competition intensity λ, the dispersal bias parameter σ and the overal dispersal rate τ. The data consist in a phylogeny with node heights proportional to their ages and geographical locations for evety taxon in this tree. An important assumption of the model is that each node in the phylogeny corresponds to a speciation *and* a dispersal event. As a consequence, this model does not authorize a given taxon to occupy more than one locations. Note however that the converse is not true: a given location can be occupied by several different taxa.

12.2.1 Installing PhyloGeo

Compiling PhyloGeo is straightforward on Unix-like machines (i.e., linux and MacOS systems). PhyloGeo is not readily available for Windows machines but compilation should be easy on this system too. In the 'phyml' directory, where the 'src/' and 'doc/' directories stand, enter the following commands:

```
./configure --enable-geo;
make clean;
make;
```

This set of commands generates a binary file called `phylogeo` which can be found in the 'src/' directory.

12.2.2 Running PhyloGeo

PhyloGeo takes as input a rooted tree file in Newick format and a file with geographical locations for all the tips of the phylogeny. Here is an example of valid tree and the corresponding spatial locations just below:

Valid PhyloGeo input tree

```
(((unicaA:1.30202,unicaB:1.30202):1.34596,(((nitidaC:0.94617,(nitidaA:0.31497,
nitidaB:0.31497):0.63120):0.18955,(((mauiensisA:0.00370,mauiensisB:0.00370):0.20068,
(pilimauiensisA:0.05151,pilimauiensisB:0.05151):0.15287):0.78769,(brunneaA:0.10582,
brunneaB:0.10582):0.88625):0.14365):0.80126,(((molokaiensisA:0.03728,
molokaiensisB:0.03728):0.71371,(deplanataA:0.01814,deplanataB:0.01814):0.73285):0.34764,
((parvulaA:0.20487,parvulaB:0.20487):0.40191,(kauaiensisA:0.24276,
kauaiensisB:0.24276):0.36401):0.49186):0.83835):0.71099):1.38043,
(nihoaA:0.05673,nihoaB:0.05673):3.97168);
```

Valid PhyloGeo spatial location file

```
nihoaA                  23.062222        161.926111
nihoaB                  23.062222        161.926111
kauaiensisA             22.0644445        159.5455555
kauaiensisB             22.0644445        159.5455555
unicaA                  21.436875        158.0524305
unicaB                  21.436875        158.0524305
parvulaA                21.436875        158.0524305
parvulaB                21.436875        158.0524305
molokaiensisA           20.90532        156.6499
molokaiensisB           20.90532        156.6499
deplanataA              20.90532        156.6499
deplanataB              20.90532        156.6499
brunneaA                20.90532        156.6499
brunneaB                20.90532        156.6499
mauiensisA              20.90532        156.6499
mauiensisB              20.90532        156.6499
pilimauiensisA          20.90532        156.6499
pilimauiensisB          20.90532        156.6499
nitidaA                 19.7362                155.6069
nitidaB                 19.7362                155.6069
nitidaC                 19.7362                155.6069
```

In order to run PhyloGeo, enter the following command: `./phylogeo ./tree_file ./spatial_location_file > phylogeo_output`. PhyloGeo will then print out the sampled values of the model parameters in the file `phylogeo_output`. This file can then be used to generate the marginal posterior densities of the model parameters. In particular, evidence for competition corresponds to value of λ smaller than 1.0. Please see the original article for more information on how to interpret the model parameters.

12.2.3 Citing PhyloGeo

Ranjard, L., Welch D., Paturel M. and Guindon S. "Modelling competition and dispersal in a statistical phylogeographic framework". 2014. Systematic Biology.

12.3 PhyREX

PhyREX is a program that is part of the PhyML package which implements phylogeographic models, i.e., models that analyse geo-referenced genetic sequences so as to infer the rate of diffusion of lineages in space along with other parameters of biological interest.

Previous versions of PhyREX implemented the **spatial Λ-Fleming-Viot model** [54–59] or ΛV for short. This model can be thought of as a structured-coalescent where space is considered as continuous rather than organized into separate demes. Under the ΛV model, the spatial distribution of individuals in a population is uniform and does not change during the course of evolution, as opposed to other models such as the very popular isolation by distance model proposed by Wright and Malécot. The ΛV model does not suffer from the "pain in the torus" [60] and clumps of individuals with increasing densities do not arise. Also, estimates of migration parameters are not sensitive to variation in sampling intensities across regions, unlike "mugration" or "discrete trait analysis" models [61]. PhyREX implements an original data augmentation technique embedded in a Bayesian sampler to estimate two important biological parameters: the neighorhood size ($\mathcal{N}$) and the dispersal intensity (σ^2). These two parameters are closely related to the effective size of the population of interest per unit area since $\mathcal{N}$ is defined as follows: $\mathcal{N} := 4\pi\rho_e\sigma^2$, where ρ_e is the effective population density.

Instead of the ΛV process, more recent releases of PhyREX implement various versions of the **Relaxed Random Walk** (RRW) model [62]. This approach relies on a Brownian diffusion process running along the phylogeny that governs the random fluctuations of 2d-coordinates during the course of evolution. As opposed to the ΛV model, the RRW approach is not mechanistic in the sense that it is not built on biological hypotheses about the migration and reproduction process at at the individual level. Yet, it defines a flexible framework to model the dispersal of organisms and is able, at least in principle, to capture important features of the underlying processes.

12.3.1 Installing PhyREX

Compiling PhyREX is straightforward on Unix-like machines (i.e., linux and MacOS systems). PhyREX is not readily available for Windows machines but compilation should be easy on this system too. In the 'phyml' directory, where the 'src/' and 'doc/' directories stand, enter the following commands:

```
./configure --enable-phyrex;
make clean;
make;
```

This set of commands generates a binary file called `phyrex` which can be found in the 'src/' directory.

12.3.2 Running PhyREX

Example input files for PhyREX can be found in the `examples/phyrex/` directory. PhyREX takes as input a XML file (See Section 10), in a PhyML-

compatible format. This file defines the models and various parameter settings for conducting an analysis. We give more information below about this file. The XML file gives the names of a sequence alignment file in the standard PHYLIP (or NEXUS) format along with a file providing the spatial coordinates of the corresponding sequences. An example of the coordinates file is given below:

Valid PhyREX spatial location file

```
# state.name lat lon
|SouthWest| 0 0
|NorthEast| 10 10
levosl 5.082206 4.133893
kmgcwv 5.914294 4.603446
uhfwja 4.990937 4.445124
ndmwkc 5.178017 4.442268
jpadex 3.747484 4.571090
lqcdcw 7.081925 5.133123
wnsbtg 4.164588 4.720346
ptwgnn 5.711159 4.462993
jhqdsm 3.539525 4.537706
vlnoes 4.613251 4.470530
pfrnpk 4.117791 4.489819
elwdvr 5.649958 4.824092
lptxiv 4.563302 4.005124
```

This first column in this file gives the sequence names, the second is the latitude and the third column gives the longitude. The first row in this file gives the names of the columns. It starts with the '#' character signaling a comment. The second and third rows define the limits of the population's habitat. At the moment, this habitat is assumed to be a rectangle. The position in space of that rectangle are determined by the coordinates of the bottom-left corner (`|SouthWest| 0 0`) and the top-right one (`|NorthEast| 10 10`). The '`|`' characters help identify the terms in the list of coordinate corresponding indeed to these two particular points and should thus not be omitted. The geographical limits of the habitat are considered as data under the ΛV model while they are ignored under the RRW model (although one needs to define these coordinates such that all samples are located within the corresponding rectangle). The following rows give the coordinates of each taxon. Every taxon in the sequence file should also be listed in the coordinate file.

Below is an excerpt of a XML file that can be used as input to run a PhyREX analysis. The phylogeographic model parameters are estimated using a Bayesian sampler which hyperparameters (number of samples, frequency at which samples are collected, length of the "burnin" phase, etc.) are set through the corresponding XML attributes (see 12.1.5). Below is the list of the main options :

- *Evolution of spatial coordinates along the tree.* `<spatialmodel>` component. This is the part where the RRW model is parametrized. Users need to specify what probabilistic distribution is to be used in order to model the variation of the spatial dispersal rates across lineages. Using `name="rw"` implements a "strict" random walk, i.e., the dispersal rate is the same throughout the whole tree. `name="rrw+gamma"` and `name="rrw+lognormal"` implements the relaxed random walk model using a gamma or a lognormal distribution of the relative rate of dispersal respectively. The modes of these distributions is set to one and the standard deviation is fixed to 2.0. The dispersal rate itself is a composite parameter that is estimated from the data. Each spatial dimension, corresponding to the North-South and East-West axes, has its own dispersal parameter. The prior distribution that applies to each of these two parameter is an exponential distribution with mean defined by the user through the `dispersal.prior.mean` attribute. Finally, the `sampling` attribute is either `survey` or `detection` (default). This attribute specifies the spatial sampling scheme. `detection` corresponds to the case where sampling is proportional to the underlying population spatial density (i.e., the most densely populated areas are also the most sampled ones). `survey` corresponds to the situation where sampling and population densities are decoupled, corresponding to situations where some areas of the habitat are easier to access for instance.

- *Substitution rate (a.k.a., clock rate).* `<clockrate>` component. In case all sequences were collected at (approximately) the same time, it is recommended to set the average rate of evolution to a specific value (`<clockrate value="1E-4"/>`). It is also possible to apply a lognormal prior on the clock rate with mean parameter and variance defined as in the following example: `<clockrate prior.mean="0.001" prior.var="1.E-8"/>`. With serially-sampled data, the average rate of evolution can be estimated directly from the data and setting it to a particular value or using an informative prior is not mandatory.

- *Variation of rates across lineages (a.k.a. relaxed clock).* `<lineagerates>` component. PhyREX is similar to PhyTime in the sense that substitution rates and times are separate parameters. The rates can vary across lineages (using for instance a log-normal model: `<lineagerates model="lognormal"/>`) or not (strict clock: `<lineagerates model="strictclock"/>`). Finally, it is necessary to give information about the time at which each sequence was collected.

This is done through seeting appropriate calibration constraints. From a technical perspective, these constraints apply to clades where each clade here is a single tip.

- *Tree generating model.* `<treegenerating>` component. Using `model="coalescent"` fits a coalescent model to the data. If no other options are given, Kingman's coalescent is used. Using `powgrowth="yes"` fits a coalescent where the population size varies according to a power function $N(t) = N_e(t+1)^{-g}$ where N_e is the effective population size at time $t = 0$ and g is the growth parameter, which is to be estimated from the data. Using `expgrowth="yes"` fits a coalescent where the population size varies according to an exponential function $N(t) = N_e e^{-gt}$. The prior distribution on the growth parameter g is exponential with mean set through the `neff.prior.mean` attribute value.

Example (excerpt) of PhyREX XML file

```
<phyrex run.id="example" output.file="out"
mcmc.chain.len="1E+7"
mcmc.sample.every="500"
mcmc.print.every="100"
mcmc.burnin="10000"
mutmap="no">

<!-- ‘‘Standard’’ stuff goes here, i.e., <topology>, -->
<!-- <partitionelem>, etc -->

<!-- Model of variation of spatial diffusion across lineages -->
<spatialmodel name="rrw+gamma" sampling="survey"
dispersal.prior.mean="2."/>

<!-- Lognormal prior on the rate of substitutions -->
<clockrate prior.mean="0.001" prior.var="1.E-8"/>

<!-- Model of rate variation across lineages -->
<treegenerating model="coalescent" neff.prior.mean="10.0"
powgrowth="yes"/>

<-- File where 2-D spatial coordinates are found -->
<coordinates id="coordinates" file.name="./usa_coord.txt"/>

<!-- Set the date for each tip. Useful in serial sample -->
<!-- analysis -->
<clade id="clad1">
<taxon value="CY130177|South_Carolina|12_13|H1N1"/>
</clade>

<!-- Clade 1 (made of a single sequence) has time set to 0 -->
<calibration id="cal1">
  <lower>0</lower>
  <upper>0</upper>
  <appliesto clade.id="clad1"/>
</calibration>


<clade id="clad2">
<taxon value="KF647978|Idaho|12_13|H1N1"/>
</clade>

<!-- Clade 2 has time set to 10 -->
<calibration id="cal2">
  <lower>10</lower>
  <upper>10</upper>
  <appliesto clade.id="clad2"/>
</calibration>

<!-- More calibration info goes here. At least, one -->
<!-- calibrated clade per tip. -->

```

PhyREX generates correlated samples from the posterior distribution of the ΛV model parameters. The estimated distributions and related summary statistics can be monitored during the analysis using the MCMC vizualization software Icylog or Tracer. Most of the names of the sampled parameters are self-explanatory but some of them (listed below) require more explanations:

- `lnPost`: log-posterior density.

- `lnAlgn`: logarithm of the probability of the sequence alignment given the current tree.
- `lnSpac`: logarithm of the probability density of the location data given the current tree.
- `lnRate`: logarithm of the probability density of the substitution rates along the tree.
- `lnTime`: logarithm of the probability density of the tree given the tree-generating model.
- `substRate`: substitution rate.
- `siqSqLon`: diffusion rate along the East-West axis.
- `siqSqLat`: diffusion rate along the North-South axis.
- `nEff`: effective population size.
- `growth`: coalescent population growth parameter.
- `dispDistAlt`: effective diffusion rate (average number of kilometers covered in one unit of time).
- `rootTime`: calendar time corresponding to the MRCA node.
- `rootLon`: longitude of the root node location.
- `rootLat`: latitude of the root node location.

If you are unsure about the precise definition of one of these parameters or have any questions regarding how to interpret the results returned by PhyREX, please do not hesitate to contact me (guindon@lirmm.fr).

12.3.3 Citing PhyREX

The "default citation" is:

- Guindon S., H Guo, D Welch. "Demographic inference under the coalescent in a spatial continuum", *Theoretical Population Biology*, 2016, 111:43-50.

13 Recommendations on program usage

13.1 PhyML

The choice of the tree searching algorithm among those provided by PhyML is generally a tough one. The fastest option relies on local and simultaneous modifications of the phylogeny using NNI moves. More thorough explorations of the space of topologies are also available through the SPR options. As these two classes of tree topology moves involve different computational burdens, it is important to determine which option is the most suitable for the type of data set or analysis one wants to perform. Below is a list of recommendations for typical phylogenetic analyses.

1. *Single data set, unlimited computing time.* The best option here is probably to use a SPR search (i.e., straight SPR of best of SPR and NNI). If the focus is on estimating the relationships between species, it is a good idea to use more than one starting tree to decrease the chance of getting stuck in a local maximum of the likelihood function. Using NNIs is appropriate if the analysis does not mainly focus on estimating the evolutionary relationships between species (e.g. a tree is needed to estimate the parameters of codon-based models later on). Branch supports can be estimated using bootstrap and approximate likelihood ratios.

2. *Single data set, restricted computing time.* The three tree searching options can be used depending on the computing time available and the size of the data set. For small data sets (i.e., < 50 sequences), NNI will generally perform well provided that the phylogenetic signal is strong. It is relevant to estimate a first tree using NNI moves and examine the reconstructed phylogeny in order to have a rough idea of the strength of the phylogenetic signal (the presence of small internal branch lengths is generally considered as a sign of a weak phylogenetic signal, specially when sequences are short). For larger data sets (> 50 sequences), a SPR search is recommended if there are good evidence of a lack of phylogenetic signal. Bootstrap analysis will generally involve large computational burdens. Estimating branch supports using approximate likelihood ratios therefore provides an interesting alternative here.

3. *Multiple data sets, unlimited computing time.* Comparative genomic analyses sometimes rely on building phylogenies from the analysis of a large number of gene families. Here again, the NNI option is the most

relevant if the focus is not on recovering the most accurate picture of the evolutionary relationships between species. Slower SPR-based heuristics should be used when the topology of the tree is an important parameter of the analysis (e.g., identification of horizontally transferred genes using phylogenetic tree comparisons). Internal branch support is generally not a crucial parameter of the multiple data set analyses. Using approximate likelihood ratio is probably the best choice here.

4. *Multiple data sets, limited computing time.* The large amount of data to be processed in a limited time generally requires the use of the fastest tree searching and branch support estimation methods Hence, NNI and approximate likelihood ratios rather than SPR and non-parametric bootstrap are generally the most appropriate here.

Another important point is the choice of the substitution model. While default options generally provide acceptable results, it is often warranted to perform a pre-analysis in order to identify the best-fit substitution model. This pre-analysis can be done using popular software such as Modeltest [63] or ProtTest [64] for instance. These programs generally recommend the use of a discrete gamma distribution to model the substitution process as variability of rates among sites is a common feature of molecular evolution. The choice of the number of rate classes to use for this distribution is also an important one. While the default is set to four categories in PhyML, it is recommended to use larger number of classes if possible in order to best approximate the patterns of rate variation across sites [65]. Note however that run times are directly proportional to the number of classes of the discrete gamma distribution. Here again, a pre-analysis with the simplest model should help the user to determine the number of rate classes that represents the best trade-off between computing time and fit of the model to the data.

13.2 PhyTime

Analysing a data set using PhyTime should involve three steps based on the following questions: (1) do the priors seem to be adequate (2) can I use the fast approximation of the likelihood and (3) how long shall I run the program for? I explain below how to provide answers to these questions.

- *Are the priors adequate?* Bayesian analysis relies on specifiying the joint prior density of model parameters. In the case of node age estimation, these priors essentially describe how rates of substitution vary across lineages and the probabilistic distribution that node ages have when ignoring the information provided by the genetic sequences. These

priors vary from tree to tree. It is therefore essential to check the adequacy of priors for each user-defined input tree. In order to do so, PhyTime needs to be run with the `--no_data` option. When this option is required, the sequence data provided as input will be ignored and the rest of the analysis will proceed normally. The prior distribution of model parameters, essentially edge rates and node heights, can then be checked using the program Tracer as one would do for the standard 'posterior' analysis.

- *Can I use the fast approximation to the likelihood?* The suface of the log-likelihood function can be approximated using a multivariate normal density. This technique is saving very substantial amounts of computation time. However, like most approximations, there are situations where it does not provide a good fit to the actual function. This usually happens when the phylogeny displays a lot of short branches, i.e., the signal conveyed by the sequences is weak. It is therefore important to first check whether using the approximate likelihood is reasonable. In order to do so, it is recommended to first run the program without the approximation, i.e., using the default settings. Once the minimum value of the ESS of node ages (the last column on the right of the standard output) has reached 40-50, open the `phytime.XXXX` output file with Tracer and examine the correlation between the exact and approximate likelihood values. If the correlation is deemed to be good enough, PhyTime can be re-run using the `--fast_lk` option, which uses the fast normal approximation to the likelihood function.

- *How long shall I run the program for?* PhyTime should be run long enough such that the ESS of each parameter is 'large enough'. The last column on the right handside of the standard output gives the minimum ESS across all internal node heights. It is recommended to run the program so that this number reaches at least 100.

14 Frequently asked questions

1. *PhyML crashes before reading the sequences. What's wrong ?*

   - The format of your sequence file is not recognized by PhyML. See Section 7
   - The carriage return characters in your sequence files are not recognized by PhyML. You must make sure that your sequence file

is a plain text file, with standard carriage return characters (i.e., corresponding to "\n", or "\r")

2. *The program crashes after reading the sequences. What's wrong ?*

   - You analyse protein sequences and did not enter the `-d aa` option in the command-line.
   - The format of your sequence file is not recognized by PhyML. See Section 7

3. *Does PhyML handle outgroup sequences ?*

   - Yes, it does. Outgroup taxa are identified by adding the '*' sign at the end of each corresponding sequence name (see Section 7.1.2)

4. *Does PhyML estimate clock-constrained trees ?*

   - No, the PhyML program does not estimate clock-contrained trees. One can however use the program PhyTime to perform such analysis but the tree topology will not be estimated.

5. *Can PhyML analyse partitioned data, such as multiple gene sequences ?*

   - We are currently working on this topic. Future releases of the program will provide options to estimate trees from phylogenomic data sets, with the opportunity to use different substitution models on the different data partitions (e.g., different genes). PhyML will also include specific algorithms to search the space of tree topologies for this type of data.

15 Acknowledgements

The development of PhyML since 2000 has been supported by the Centre National de la Recherche Scientifique (CNRS) and the Ministère de l'Éducation Nationale.

References

[1] Guindon, S. & Gascuel, O. A simple, fast and accurate algorithm to estimate large phylogenies by maximum likelihood. *Systematic Biology* **52**, 696–704 (2003).

[2] Stamatakis, A. RAxML-VI-HPC: Maximum likelihood-based phylogenetic analyses with thousands of taxa and mixed models. *Bioinformatics* **22**, 2688–2690 (2006).

[3] Zwickl, D. *Genetic algorithm approaches for the phylogenetic analysis of large biological sequence datasets under the maximum likelihood criterion*. Ph.D. thesis, The University of Texas at Austin (2006).

[4] Ayres, D. L. *et al.* Beagle: an application programming interface and high-performance computing library for statistical phylogenetics. *Systematic Biology* **61**, 170–173 (2012).

[5] Jukes, T. & Cantor, C. Evolution of protein molecules. In Munro, H. (ed.) *Mammalian Protein Metabolism*, vol. III, chap. 24, 21–132 (Academic Press, New York, 1969).

[6] Kimura, M. A simple method for estimating evolutionary rates of base substitutions through comparative studies of nucleotide sequences. *Journal of Molecular Evolution* **16**, 111–120 (1980).

[7] Felsenstein, J. Evolutionary trees from DNA sequences: a maximum likelihood approach. *Journal of Molecular Evolution* **17**, 368–376 (1981).

[8] Felsenstein, J. *PHYLIP (PHYLogeny Inference Package) version 3.6a2* (Distributed by the author, Department of Genetics, University of Washington, Seattle, 1993).

[9] Hasegawa, M., Kishino, H. & Yano, T. Dating of the Human-Ape splitting by a molecular clock of mitochondrial-DNA. *Journal of Molecular Evolution* **22**, 160–174 (1985).

[10] Tamura, K. & Nei, M. Estimation of the number of nucleotide substitutions in the control region of mitochondrial DNA in humans and chimpanzees. *Molecular Biology and Evolution* **10**, 512–526 (1993).

[11] Lanave, C., Preparata, G., Saccone, C. & Serio, G. A new method for calculating evolutionary substitution rates. *Journal of Molecular Evolution* **20**, 86–93 (1984).

[12] Tavaré, S. Some probabilistic and statistical problems on the analysis of DNA sequences. *Lectures on Mathematics in the Life Sciences* **17**, 57–86 (1986).

[13] Le, S. & Gascuel, O. An improved general amino-acid replacement matrix. *Molecular Biology and Evolution* **25**, 1307–1320 (2008).

[14] Whelan, S. & Goldman, N. A general empirical model of protein evolution derived from multiple protein families using a maximum-likelihood approach. *Molecular Biology and Evolution* **18**, 691–699 (2001).

[15] Dayhoff, M., Schwartz, R. & Orcutt, B. A model of evolutionary change in proteins. In Dayhoff, M. (ed.) *Atlas of Protein Sequence and Structure*, vol. 5, 345–352 (National Biomedical Research Foundation, Washington, D. C., 1978).

[16] Jones, D., Taylor, W. & Thornton, J. The rapid generation of mutation data matrices from protein sequences. *Computer Applications in the Biosciences (CABIOS)* **8**, 275–282 (1992).

[17] Henikoff, S. & Henikoff, J. Amino acid substitution matrices from protein blocks. *PNAS* **89**, 10915–10919 (1992).

[18] Adachi, J. & Hasegawa, M. MOLPHY version 2.3. programs for molecular phylogenetics based on maximum likelihood. In Ishiguro, M. *et al.* (eds.) *Computer Science Monographs*, 28 (The Institute of Statistical Mathematics, Tokyo, 1996).

[19] Dimmic, M., Rest, J., Mindell, D. & Goldstein, D. rtREV: an amino acid substitution matrix for inference of retrovirus and reverse transcriptase phylogeny. *Journal of Molecular Evolution* **55**, 65–73 (2002).

[20] Adachi, J., P., W., Martin, W. & Hasegawa, M. Plastid genome phylogeny and a model of amino acid substitution for proteins encoded by chloroplast DNA. *Journal of Molecular Evolution* **50**, 348–358 (2000).

[21] Kosiol, C. & Goldman, N. Different versions of the Dayhoff rate matrix. *Molecular Biology and Evolution* **22**, 193–199 (2004).

[22] Muller, T. & Vingron, M. Modeling amino acid replacement. *Journal of Computational Biology* **7**, 761–776 (2000).

[23] Cao, Y. *et al.* Conflict among individual mitochondrial proteins in resolving the phylogeny of eutherian orders. *Journal of Molecular Evolution* **47**, 307–322 (1998).

[24] Gascuel, O. BioNJ: an improved version of the NJ algorithm based on a simple model of sequence data. *Molecular Biology and Evolution* **14**, 685–695 (1997).

[25] Anisimova, M., Gil, M., Dufayard, J., Dessimoz, C. & Gascuel, O. Survey of branch support methods demonstrates accuracy, power, and robustness of fast likelihood-based approximation schemes. *Systematic Biology* **60**, 685–699 (2011).

[26] Anisimova, M. & Gascuel, O. Approximate likelihood-ratio test for branches: a fast, accurate, and powerful alternative. *Systematic Biology* **55**, 539–552 (2006).

[27] Lemoine, F. *et al.* Renewing Felsenstein's phylogenetic bootstrap in the era of big data. *Nature* **556**, 452–456 (2018).

[28] Lutteropp, S., Kozlov, A. M. & Stamatakis, A. A fast and memory-efficient implementation of the transfer bootstrap. *Bioinformatics* **36**, 2280–2281 (2020).

[29] Mirsky, A., Kazandjian, L. & Anisimova, M. Antibody-specific model of amino acid substitution for immunological inferences from alignments of antibody sequences. *Molecular Biology and Evolution* **32**, 806–819 (2015).

[30] Dang, C. C., Le, Q. S., Gascuel, O. & Le, V. S. FLU, an amino acid substitution model for influenza proteins. *BMC Evolutionary Biology* **10**, 99 (2010).

[31] Minh, B. Q., Dang, C. C., Vinh, L. S. & Lanfear, R. QMaker: fast and accurate method to estimate empirical models of protein evolution. *Systematic Biology* **70**, 1046–1060 (2021).

[32] Rota-Stabelli, O., Yang, Z. & Telford, M. J. MtZoa: a general mitochondrial amino acid substitutions model for animal evolutionary studies. *Molecular Phylogenetics and Evolution* **52**, 268–272 (2009).

[33] Le, S. Q., Dang, C. C. & Gascuel, O. Modeling protein evolution with several amino acid replacement matrices depending on site rates. *Molecular Biology and Evolution* **29**, 2921–2936 (2012).

[34] Le, S. Q. & Gascuel, O. Accounting for solvent accessibility and secondary structure in protein phylogenetics is clearly beneficial. *Systematic Biology* **59**, 277–287 (2010).

[35] Posada, D. & Crandall, K. Selecting models of nucleotide substitution: an application to human immunodeficiency virus 1 (hiv-1). *Molecular Biology and Evolution* **18**, 897–906 (2001).

[36] Soubrier, J. *et al.* The influence of rate heterogeneity among sites on the time dependence of molecular rates. *Molecular Biology and Evolution* (2012).

[37] Guindon, S. From trajectories to averages: an improved description of the heterogeneity of substitution rates along lineages. *Systematic Biology* **62**, 22–34 (2013).

[38] Lefort, V., Longueville, J.-E. & Gascuel, O. SMS: Smart model selection in PhyML. *Molecular Biology and Evolution* **34**, 2422–2424 (2017).

[39] Maddison, D., Swofford, D. & Maddison, W. NEXUS: an extensible file format for systematic information. *Systematic Biology* **46**, 590–621 (1997).

[40] Oliva, A. *et al.* Accounting for ambiguity in ancestral sequencereconstruction. *Bioinformatics* https://www.biorxiv.org/content/early/2018/09/05/409029 (2019).

[41] Yang, Z. Maximum likelihood phylogenetic estimation from DNA sequences with variable rates over sites: approximate methods. *Journal of Molecular Evolution* **39**, 306–314 (1994).

[42] Yang, Z. Likelihood ratio tests for detecting positive selection and application to primate lysozyme evolution. *Molecular Biology and Evolution* **15**, 568–573 (1998).

[43] Yang, Z. & Nielsen, R. Estimating synonymous and nonsynonymous substitution rates under realistic evolutionary models. *Molecular Biology and Evolution* **17**, 32–43 (2000).

[44] Yang, Z. & Nielsen, R. Codon-substitution models for detecting molecular adaptation at individual sites along specific lineages. *Molecular Biology and Evolution* **19**, 908–917 (2002).

[45] Guindon, S., Rodrigo, A., Dyer, K. & Huelsenbeck, J. Modeling the site-specific variation of selection patterns along lineages. *PNAS* **101**, 12957–12962 (2004).

[46] Degnan, J. & Rosenberg, N. Gene tree discordance, phylogenetic inference and the multispecies coalescent. *Trends in Ecology & Evolution* **24**, 332–340 (2009).

[47] Liu, L. BEST: Bayesian estimation of species trees under the coalescent model. *Bioinformatics* **24**, 2542–2543 (2008).

[48] Kubatko, L., Carstens, B. & Knowles, L. STEM: species tree estimation using maximum likelihood for gene trees under coalescence. *Bioinformatics* **25**, 971–973 (2009).

[49] Heled, J. & Drummond, A. Bayesian inference of species trees from multilocus data. *Molecular Biology and Evolution* **27**, 570–580 (2010).

[50] Matsen, F. & Steel, M. Phylogenetic mixtures on a single tree can mimic a tree of another topology. *Systematic Biology* **56**, 767–775 (2007).

[51] Shimodaira, H. & Hasegawa, M. Multiple comparisons of log-likelihoods with applications to phylogenetic inference. *Molecular Biology and Evolution* **16**, 1114–1116 (1999).

[52] Lartillot, N. & Philippe, H. A bayesian mixture model for across-site heterogeneities in the amino-acid replacement process. *Molecular Biology and Evolution* **21**, 1095–1109 (2004).

[53] Guindon, S. Accounting for calibration uncertainty: Bayesian molecular dating as a "doubly intractable" problem. *Systematic Biology* **67**, 651–661 (2018).

[54] Etheridge, A. M. Drift, draft and structure: some mathematical models of evolution. *Banach Center Publ.* **80**, 121–144 (2008).

[55] Berestycki, N., Etheridge, A. & Hutzenthaler, M. Survival, extinction and ergodicity in a spatially continuous population model. In *Markov Proc. Rel. Fields* (2009).

[56] Barton, N., Etheridge, A. & Véber, A. A new model for evolution in a spatial continuum. *Electronic Journal of Probability* **15** (2010).

[57] Barton, N. H., Kelleher, J. & Etheridge, A. M. A new model for extinction and recolonization in two dimensions: quantifying phylogeography. *Evolution* **64**, 2701–2715 (2010).

[58] Véber, A. & Wakolbinger, A. The spatial Lambda-Fleming-Viot process: an event-based construction and a lookdown representation. *Annales de l'Institut Henri Poincaré* in press (2014).

[59] Barton, N., Etheridge, A. & Véber, A. Modelling evolution in a spatial continuum. *Journal of Statistical Mechanics: Theory and Experiment* **2013**, P01002 (2013).

[60] Felsenstein, J. A pain in the torus: some difficulties with models of isolation by distance. *American Naturalist* 359–368 (1975).

[61] Lemey, P., Rambaut, A., Drummond, A. J. & Suchard, M. A. Bayesian phylogeography finds its roots. *PLoS Comput Biol* **5**, e1000520 (2009).

[62] Lemey, P., Rambaut, A., Welch, J. J. & Suchard, M. A. Phylogeography takes a relaxed random walk in continuous space and time. *Molecular Biology and Evolution* **27**, 1877–1885 (2010). Publisher: SMBE.

[63] Posada, D. & Crandall, K. Modeltest: testing the model of DNA substitution. *Bioinformatics* **14**, 817–918 (1998).

[64] Abascal, F., Zardoya, R. & Posada, D. Prottest: selection of best-fit models of protein evolution. *Bioinformatics* **21**, 2104–2105 (2005).

[65] Galtier, N. & Jean-Marie, A. Markov-modulated Markov chains and the covarion process of molecular evolution. *Journal of Computational Biology* **11**, 727–733 (2004).

Index

κ, 12, 19
*BEAST, 39

ancestral reconstruction, 32

BEAST, 69
BEST, 39
binary characters, 27
BioNJ, 14
bootstrap, 16
    Bayesian, 17
    parallel, 8, 23
    transfer, 16
bug, 6

CAT model, 57
command-line options
    `--constraint_file`, 23
command-line options
    `--aa_rate_file`, 18
    `--alpha`, 19
    `--ancestral`, 23
    `--bayesian_bootstrap`, 17
    `--bootstrap`, 16
    `--codpos`, 20
    `--constrained_lens`, 23
    `--data_type`, 15
    `--free_rates`, 19
    `--il`, 20
    `--inputtree`, 20
    `--input`, 15
    `--json_trace`, 21
    `--leave_duplicates`, 23
    `--model`, 17
    `--multiple`, 16
    `--n_rand_starts`, 21
    `--nclasses`, 19
    `--no_colalias`, 22
    `--no_memory_check`, 22
    `--pars`, 16
    `--pinv`, 19
    `--print_site_lk`, 21
    `--print_trace`, 21
    `--quiet`, 23
    `--r_seed`, 21
    `--rand_start`, 21
    `--run_id`, 22
    `--search`, 20
    `--sequential`, 16
    `--sms_criterion`, 22
    `--sms_only`, 22
    `--sms`, 22
    `--struct_chi`, 22
    `--struct_file`, 22
    `--struct_fix_props`, 22
    `--struct_model`, 22
    `--struct_props`, 22
    `--tbe`, 16
    `--threads`, 21
    `--ts/tv`, 19
    `--use_median`, 19
    `--xml`, 23
    `-f`, 18
    `-o`, 20
compilation, 6
covarion, 20
custom models, 18

data partitions, 37, 44

FASTA, 27
FreeRate, 51
frequencies
    amino-acid, 18
    nucleotide, 18

gamma distribution (discrete)
    mean vs. median, 12, 19

number of categories, 12, 19
shape parameter, 12, 19
GARLI, 6

Icylog, 69
input tree, 20
interleaved, 25
invariable sites, 12, 19

LG4M, 34
LG4X, 34
lg4x, 54
likelihood
print site likelihood, 21

mixture models, 44
mixture models, 36
protein, 34
model selection, 22, 33
MPEE, 32
MPI, 8, 23
MRCA, 66
multiple data sets, 16, 30
multiple-gene analysis, 37
multithreading, 7, 21, 24

Newick format, 29
NEXUS, 25
NNI, 20

OpenMP, 7, 24
optimisation
substitution parameters, 20
topology, 20

PAML, 12, 30
partitionned analysis, 37, 44
PHYLIP, 12, 25
PhyloGeo, 73
PhyREX, 74
PhyTime, 62
calibration, 63
clade, 64
lineagerates, 63
proportion of invariants, 12, 19

random number, 21
random tree, 21
RAxML, 6
relaxed clock (model), 77
RRW (model), 75
run ID, 11, 22

sequence format
interleaved, 16
sequential, 16
sequential, 25
SMS, 33
SPR, 20
stationary frequencies, 18
STEM, 39
structure models, 35
substitution models
amino acids, 17
DNA, 17

tests, 7
Tracer, 69
ts/tv ratio, 12, 19

user tree, 20

XML, 36
XML options
`branchlengths` component, 50
`equfreqs` component, 49
`mixtureelem` component, 52
`partitionelem` component, 52
`phyml` component, 46
`ratematrices` component, 48
`siterates` component, 50
`topology` component, 47